

MASTER'S THESIS

Thesis submitted in fulfillment of the requirements for the degree of Master of Science in Engineering at the University of Applied Sciences Technikum Wien - Master Artificial Intelligence & Data Science

Real-Time Market Microstructure Modeling with Temporal Graph Neural Networks

By: Vitalii Novikov, BSc
Student Number: 2410854011

Supervisor: Dr. Rosana de Oliveira Gomes, Senior Lecturer/Researcher

Vienna, 30.05.2026

Declaration of Authenticity

“As author and creator of this work to hand, I confirm with my signature knowledge of the relevant copyright regulations governed by higher education acts (see Urheberrechtsgesetz/ Austrian copyright law as amended as well as the Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I hereby declare that I completed the present work independently and according to the rules currently applicable at the UAS Technikum Wien and that any ideas, whether written by others or by myself, have been fully sourced and referenced. I am aware of any consequences I may face on the part of the degree program director if there should be evidence of missing autonomy and independence or evidence of any intent to fraudulently achieve a pass mark for this work (see Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I further declare that up to this date I have not published the work to hand nor have I presented it to another examination board in the same or similar form. I affirm that the version submitted matches the version in the upload tool.”

V. Novikov

Vienna, 30 May 2026

eIDAS Qualified Electronic Signature



Vienna, 30.05.2026

Place, Date

Digital Signature

Abstract

This thesis compares graph-based neural architectures for short-horizon limit order book prediction under a common, deployment-oriented benchmark. The empirical setting is a cryptocurrency limit order book dataset containing Cardano (ADA), Bitcoin (BTC), and Ethereum (ETH) snapshots at five-minute, one-minute, and one-second resolution. ETH is the target asset, while ADA and BTC provide relational market context. The study compares three model families: a single-graph baseline ([basegraph](#)), a multi-relation graph architecture ([multigraph](#)), and a stateful recurrent graph architecture ([memorygraph](#)). Each family is tested with a convolution-style graph operator and a message-passing neural network operator, producing eighteen primary benchmark configurations (3 frequencies, 3 architecture families, 2 operators).

The thesis is a controlled benchmark, not a production trading-system claim. All models share the same asset universe, feature construction, relation-state construction, triple-barrier target logic, multi-task output heads, walk-forward validation, final holdout interval, thresholding logic, and event-based backtest. This design makes architectural comparison more reliable because the models differ mainly in how they encode graph, relation, and temporal information. The central evaluation distinction is between ‘[last-fold-M](#)’, the final chronological cross-validation model used as the main deployment-oriented reference, and ‘[final-refit-M](#)’, a larger-sample refit state used as an informative robustness comparison.

The strongest [last-fold-M](#) result at both 5min and 1min resolution is obtained by [base-gnn-conv](#). In terms of log-return Profit and loss (PnL), at 5min, it achieves [net_pnl](#) = 0.020356 over 26 trades. At 1min, it achieves [net_pnl](#) = 0.020094 over 132 trades. At 1sec, all models remain negative after transaction costs. The 1sec experiments are still informative because several models produce positive [gross_pnl](#); the problem is that the gross edge is consumed by excessive turnover and the round-trip cost proxy. The clearest example is [memory-gnn-conv](#), which produces the largest 1sec gross signal but ends with strongly negative net PnL because it executes 5251 trades.

The thesis concludes that, under this controlled entry-model benchmark, the simpler [basegraph](#) architecture is the most reliable family. More complex relation-preserving and memory-based models provide useful diagnostics, especially for gross signal extraction and ranking quality, but they do not establish a robust post-cost advantage. Overall, the findings indicate that the central challenge in this setting is translating short-horizon microstructure signals into sufficiently selective, stable, and cost-aware trading decisions.

Keywords: limit order book; market microstructure; graph neural networks; temporal graph learning; walk-forward validation; triple-barrier labels; transaction-cost-aware backtesting

Table of Contents

Real-Time Market Microstructure Modeling with Temporal Graph Neural Networks.....	0
List of Abbreviations and Notation.....	5
1. Introduction.....	7
1.1. Motivation of the Topic.....	7
1.2. Research Gap and Thesis Scope.....	9
1.3. Research Aim.....	10
1.4. Research Questions.....	10
1.5. Hypotheses.....	11
1.6. Thesis Contribution.....	12
2. Literature Background.....	13
2.1. Market Microstructure and Limit Order Book Prediction.....	13
2.2. Deep Learning for Limit Order Books.....	13
2.3. Graph Neural Networks and Message Passing.....	14
2.4. Temporal and Dynamic Graph Learning.....	14
2.5. Financial Graph Learning.....	15
2.6. Evaluation, Backtesting, and Leakage Control.....	15
3. Data and Methodology.....	16
3.1. Data Source and Study Universe.....	16
3.2. Graph Input Representation.....	17
3.3. Node Features.....	18
3.4. Relation States and Edge Features.....	20
3.5. Scaling and Leakage Control.....	21
3.6. Frequency-Specific Experimental Regimes.....	22
3.7. Target Construction and Shared Learning Objective.....	23
3.8. Common Entry-Model Backtest.....	25
3.9. Validation Design and Deployment-Oriented Model States.....	26
3.10. Metrics.....	27
3.11. Fair-Comparison Principle.....	28
4. Model Families.....	29
4.1. Shared Architectural Conventions.....	29
4.2. The basegraph Family.....	30
4.3. The multigraph Family.....	32
4.4. The memorygraph Family.....	33
4.5. Summary of Model Families.....	35
5. Results.....	36
5.1. Benchmark Overview.....	36
5.2. Frequency-Specific Results.....	40
5.3. Answer to RQ1: Which Graph Family Performs Best?.....	42
5.4. Answer to RQ2: How Important Is the Conv-versus-MPNN Operator Choice?.....	43
5.5. Answer to RQ3: How Does Temporal Resolution Affect Relation and Memory Mechanisms?.....	43
5.6. Answer to RQ4: Are Conclusions Stable Between last-fold-M and final-refit-M?.....	44

5.7. Hypothesis Assessment.....	46
5.8. Summary of Research Question Answers.....	48
5.9. Summary of Hypothesis Assessment.....	48
6. Discussion.....	49
6.1. Main Findings by Research Question.....	49
6.2. Comparison with Previous Work.....	49
6.3. Scientific Implications.....	50
6.4. Practical and Deployment Implications.....	50
6.5. Limitations, Weaknesses, and Sources of Bias.....	51
7. Conclusions and Future Research.....	52
7.1. Overall Conclusion.....	52
7.2. Future Research.....	52
References.....	54
List of Figures.....	56
List of Tables.....	57
Documentation table of AI-based tools.....	58
Appendix A. Model Configurations.....	59
Appendix B. Final Holdout Alignment Table.....	60
Appendix C. Additional Benchmark Tables.....	61

List of Abbreviations and Notation

This thesis uses several implementation-level labels for model families, model states, graph operators, prediction heads, and trading metrics. Coloured words are supposed to help easily distinguish thesis-critical terms. The guide below is provided as a quick reference for readers.

Terminology and notation guide.

Term	Category	Meaning in this thesis	Important note
LOB	General acronym	Limit order book: the visible buy and sell quotes and depths around the current price.	The data are snapshot-based, not full exchange message streams.
GNN		Graph neural network. A neural architecture that updates node representations using relational structure.	Used here for a three-asset crypto graph.
PnL		Profit and loss.	PnL is reported as cumulative log-return-style trade performance.
ADA, BTC, ETH	Asset universe	The three assets in the graph universe.	Ethereum (ETH) is the target asset; Cardano (ADA) and Bitcoin (BTC) provide relational context.
5min, 1min, 1sec	Data regime	Frequency regimes used in the benchmark.	5min and 1min are strict shared-task regimes; 1sec is a frequency-adapted high-frequency stress test.
price	Market term	The asset price level represented by the midpoint price in the LOB snapshot.	Used to compute returns and price-dependence states.
return		Log price change over a bar or over the realized event horizon. Formula (3)	Node features include one-bar returns; PnL uses realized event returns.
spread	Market microstructure term	Difference between best ask and best bid, scaled by price when used as a feature. (4)	Wider spread usually implies higher trading friction and lower liquidity.
Flow (order-flow)		Snapshot-level buy and sell activity summaries. Formula (7)	The model uses log buy/sell flow and buy-sell flow imbalance.
depth		Visible quantity available on bid and ask sides across book levels. Formulas (8) (9)	The benchmark uses 15 bid and 15 ask levels, with near/far depth aggregates.
liquidity		Ease and cost of trading near the current price. Formula (17)	In the relation state, wider spread lowers liquidity while depth and book shape add context.
triple-barrier	General for financial ML target construction	Labelling rule that follows the future ETH price path until the upper barrier, lower barrier, or vertical time barrier is reached.	It supplies trade relevance, direction, event return, exit type, and time-to-exit labels for further supervised learning.
trade activation	Trading decision term	Binary decision that an eligible event should become an opened position rather than a skipped event.	The trade head produces this decision before the direction head chooses long or short exposure.
trading direction		The predicted side of ETH exposure: long for an expected positive ETH move and short for an expected negative ETH move.	The direction head makes this decision.
basegraph (single-graph)	Model family	Single-graph baseline family with early relation fusion.	Full benchmark labels use the prefix base-gnn-* .
multigraph (multi-relation)		Multi-relation graph family that preserves relation-specific pathways before fusion.	Full benchmark labels use the prefix multi-gnn-* .
memorygraph (memory-relation)		Stateful recurrent graph family with node and edge memory updates.	Full benchmark labels use the prefix memory-gnn-* .

Term	Category	Meaning in this thesis	Important note
base-gnn-conv, base-gnn-mpnn	Benchmark model labels	Full benchmark configurations for the basegraph family.	The suffix shows the graph operator: Conv or MPNN.
multi-gnn-conv, multi-gnn-mpnn		Full benchmark configurations for the multigraph family.	
memory-gnn-conv, memory-gnn-mpnn		Full benchmark configurations for the memorygraph family.	
last-fold-M	Model state	Model from the last chronological walk-forward fold. Assume a gap between training data and final holdout	Primary deployment-oriented reference in the thesis. In the code represented as ‘last_CV’.
final-refit-M		Model refit on the largest available pre-holdout sample (the closest data to the final holdout)	Informative robustness comparison; it should not replace last-fold-M . In the code represented as ‘final_refit’.
‘final holdout’	Validation/evaluation split	Blind chronological evaluation interval held out from model development.	Used for the final reported benchmark.
pre-holdout		Chronological region before the final holdout.	Used for training, validation, threshold selection, and refitting.
gross_pnl	Trading metric	Sum of pre-cost directional trade returns.	Measures raw signal extraction before transaction costs.
net_pnl		Sum of post-cost trade returns.	This is the primary economic metric. In prose, it can be called net PnL.
pnl_per_trade		Average post-cost return per executed trade.	Helps separate a few high-quality trades from many low-margin trades.
n_trades		Number of executed trades.	Central for interpreting cost drag and turnover.
trade_rate		Fraction of eligible events that become executed trades.	A higher trade rate is not automatically better.
sign_accuracy	Diagnostic metric	Fraction of executed trades for which the predicted side matches the realized event direction.	Directional correctness does not guarantee positive net PnL.
win_rate		Fraction of executed trades with a positive net outcome	Computed after the benchmark transaction-cost proxy.
sharpe_like		Scale-free diagnostic of mean trade return relative to trade-return dispersion.	It is not an annualized Sharpe ratio.
dir_auc		AUC for the direction head.	Core decision-head ranking metric for the side-selection component.
trade_auc		AUC for the trade activation head.	Core decision-head ranking metric for the entry-activation component..
RMSE		Root mean squared error for return prediction.	Accompanying regression diagnostic.
cost_bps_unit	Cost term	Round-trip transaction-cost proxy subtracted from each trade.	In the benchmark, cost_bps_unit = 0.0003.
cost drag	Cost interpretation term	Difference between cumulative gross PnL and cumulative net PnL caused by transaction costs.	Especially important for the 1sec memorygraph results.
price-dependence state	Relation state (tho core of “edges” in graph)	Asset-level return series used to construct rolling pairwise edge features.	Edge features include correlation, beta, and mean product over lags and windows.
order-flow state		Signed buy-sell flow imbalance scaled by log turnover.	Used as one of the three relation channels for edge features.
liquidity state		Feature state built from relative spread, depth imbalance, and near/far book shape.	It is a model feature, not a full market-wide liquidity measure.

1. Introduction

1.1. Motivation of the Topic

Financial markets generate high-volume streams of event-like information. At high frequency, prices, spreads, order-flow summaries, and visible depth change faster than a human analyst can inspect them manually. The limit order book (LOB) is therefore a natural object for data science: it records the visible supply and demand around the current price and provides a structured view of short-horizon market dynamics [1], [2]. It is also a difficult prediction environment. Useful signals are weak, non-stationary, regime-dependent, and highly sensitive to latency, costs, threshold choice, and model-selection bias. These difficulties are consistent with well-known stylized facts of financial returns, including heavy tails, volatility clustering, and changing dependence structures [3]. They also appear in cryptocurrency LOBs, where empirical studies report similarities to mature electronic markets but also shallower books and relatively high liquidity costs [4].

This difficulty creates both a scientific and a practical motivation. Scientifically, LOB prediction is a demanding test case for machine learning on noisy sequential data. A model must process temporal dependence, cross-asset information, changing liquidity, and shifting dependence patterns without relying on a stationary data-generating process. Practically, a forecast is not valuable only because it predicts a future direction. It becomes valuable only if it can be translated into selective decisions after transaction costs are considered. This thesis therefore treats predictive quality, gross signal quality, and net economic value as related but distinct layers of evidence.

Recent machine learning research has shown that deep architectures can learn useful representations from LOB data. Convolutional and recurrent models learn local book structure and temporal dependence [5], [6], [7]; large-scale order-flow studies report cross-instrument regularities [8], [9], [10]; and recent LOB work has moved toward attention, transformer, survival-analysis, and topology-aware representations [11]. These results motivate representation learning, but they do not remove the need for chronological validation. In financial prediction, random splits can leak information across time, and labels based on future price paths can create overlap between neighbouring observations. For this reason, the present benchmark uses a walk-forward folds for training and evaluation and a final chronological holdout for testing rather than random cross-validation (see section 3.9). The corresponding financial machine learning validation framework is discussed in [12].

Graph-based modelling provides an additional motivation. Financial assets do not evolve independently: returns, order-flow pressure, spreads, and liquidity states can co-move, lead, lag, or diverge. A graph representation makes this relational structure explicit by representing assets as nodes and cross-asset dependence measures as edges. Static graph neural networks are suitable when relations are fixed or slowly varying [13], [14], [15], while temporal and dynamic graph learning address settings in which node states, edge states, or interaction patterns evolve over time [16], [17], [18], [19], [20]. This makes graph-based modelling plausible for cross-asset market microstructure. Figure 1.1 summarizes the conceptual pipeline studied in the thesis.

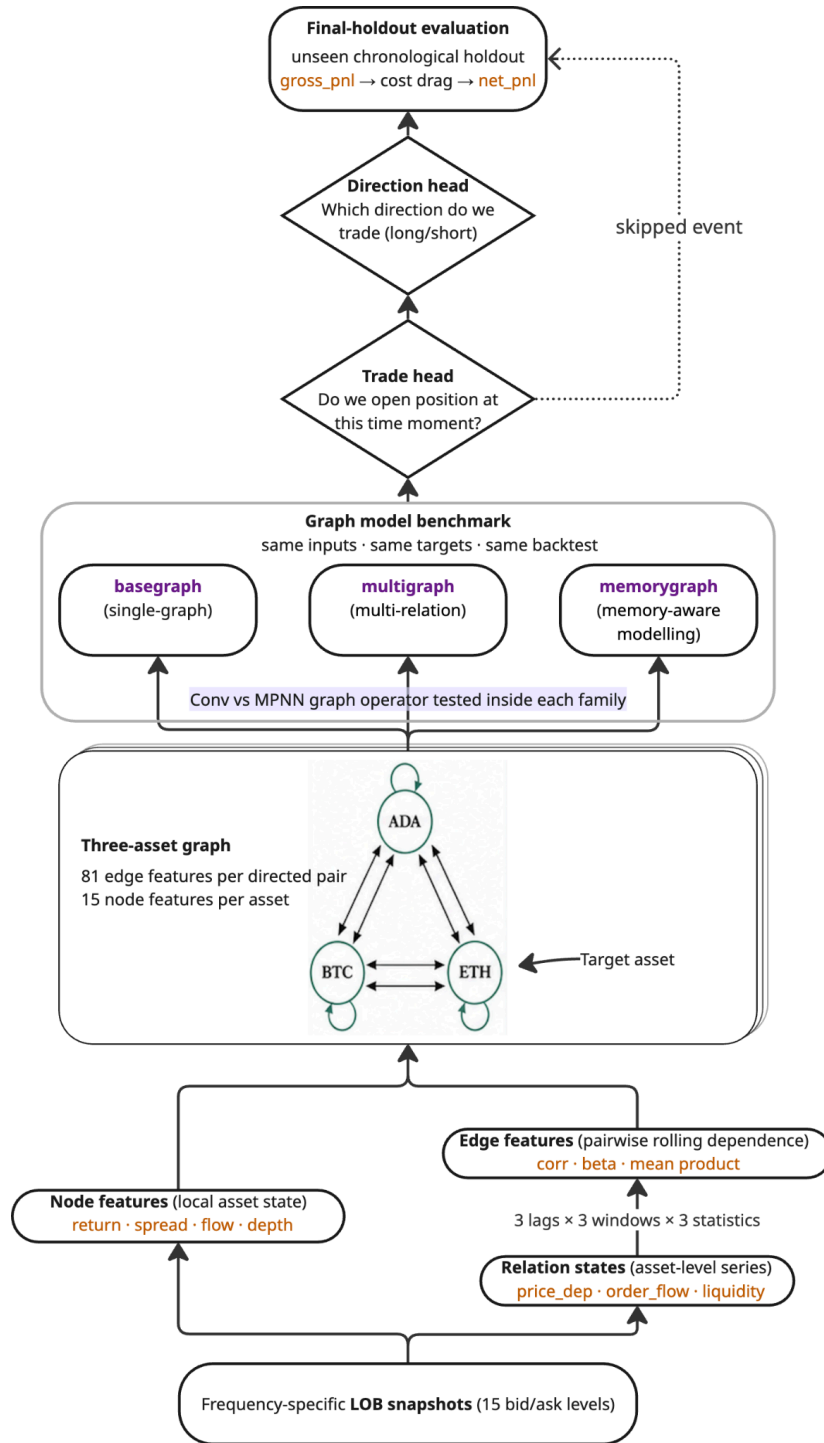


Figure 1.1 - Conceptual graph-to-trade pipeline

Figure 1.1 provides a high-level description on how the data transforms from LOB to graph-friendly input (bottom part of the picture) and on how the suggested models make a decision to trade (top part). The raw input is a frequency-specific LOB snapshot for each of the assets. At each timestamp, the snapshot is converted into a time-dependent three-asset graph state. And all these states together build a dynamic graph. In this graph, assets are nodes, directed edges represent ordered source-target asset pairs, and graph features are the numerical variables attached to nodes and edges. Node features describe each asset's local state, such as return, spread, flow, and depth (see section 3.3). Edge features summarize

rolling pairwise dependence in price, order-flow, and liquidity relation states (see section 3.4). The asset set and edge list are fixed, but their feature values change over time.

The benchmark then feeds the same graph input into three model families: basegraph, multigraph, and memorygraph. Inside each family, the Conv-versus-MPNN comparison refers to the local graph-interaction rule, not to the temporal encoder: the convolution-style (Conv-style) operator aggregates neighbouring asset representations through edge-weighted source-node projections plus edge-conditioned shifts, while the message-passing neural network (MPNN-style) operator forms richer gated messages from source-node, destination-node, and edge states. The outputs are two decision-critical heads. The trade head decides whether to open a position at the event; if it is inactive, the event is skipped. If it is active, the direction head chooses long or short ETH exposure, meaning a position that benefits from an expected positive or negative ETH move. The realized triple-barrier event supplies the common exit and return (see section 3.7). `gross_pnl` records the directional event return before transaction costs, and `net_pnl` subtracts the shared cost proxy. The final holdout applies this complete pipeline to an unseen later chronological segment.

This work studies this idea in a deliberately controlled form. It does not attempt to build a full production trading system. Instead, it compares how different graph-based architectures behave under the same entry-model benchmark, with the data, targets, output heads, validation design, thresholding logic, and event-based trading evaluation held as consistent as possible. The empirical question is therefore architectural and diagnostic: under a shared benchmark, how do single-graph, multi-relation, memory-based, Conv-style, and MPNN-style designs differ in signal extraction and post-cost performance?

1.2. Research Gap and Thesis Scope

The literature contains several relevant strands. Market microstructure explains why order-flow, liquidity, and the organization of the book matter for short-horizon price formation [1], [2]. Deep learning for LOB prediction shows that neural networks can learn from high-dimensional book states [5], [6], [7], [8], [9], [10], [11]. Graph neural networks provide tools for learning from relational data [13], [14], [15], while dynamic graph methods extend this idea to systems whose states and relations evolve over time [17], [18], [19], [20]. Financial graph learning also shows how relation structures can be useful in stock prediction and other financial tasks [21], [22].

The gap addressed here is narrower and more empirical. Many financial graph studies focus on daily or lower-frequency relations, while many LOB studies model a single instrument without explicitly representing cross-asset graph structure. This thesis examines a small but controlled crypto limit order book setting in which ADA, BTC, and ETH form a three-node graph, ETH is the target asset, and cross-asset relation states are rebuilt at 5min, 1min, and 1sec resolutions. The study focuses on whether graph family, graph operator, and temporal resolution change the usefulness of relational and memory-aware modelling under a common trading-oriented evaluation.

The scope is intentionally limited. The benchmark uses a fixed asset universe, a fixed target asset, a common triple-barrier target construction, and a shared non-overlapping event backtest. This design improves internal comparability, but it also means that the thesis evaluates entry models (rather than complete trading systems with jointly optimized execution and exit policies).

1.3. Research Aim

The aim of this thesis is to evaluate how different graph-based architectures (18 models) behave in short-horizon limit order book prediction when tested under an apples-to-apples, friction-aware benchmark.

The core object of interest is the model family. The study asks how a simple single-graph representation (**basegraph**), a multi-relation representation (**multigraph**), and a stateful memory-based representation (**memorygraph**) differ in predictive diagnostics, trading selectivity, turnover, and post-cost outcomes. A second object of interest is the graph operator: a Conv-style operator versus a MPNN-style graph operator. A third object of interest is deployment stability: whether the same conclusions hold when moving from the last chronological cross-validation state (**last-fold-M**) to a final refit state (**final-refit-M**).

1.4. Research Questions

The thesis is guided by four research questions. Figure 1.2 shows how the questions connect model family, graph operator, temporal resolution, and deployment-oriented model state. Table 1.1 then maps each question to the research method.

RQ1. Which graph family performs best under a controlled entry-model benchmark?

The first question asks whether the simpler single-graph baseline, the multi-relation graph family, or the stateful memory graph family produces the strongest final-holdout trading result when all families are evaluated under the same target construction, thresholding logic, and event-based backtest.

RQ2. How important is the Conv-versus-MPNN operator choice inside each family?

Each family is evaluated with a Conv-style operator and an MPNN-style operator. This makes it possible to distinguish the effect of the broader model family from the effect of the local graph interaction mechanism.

RQ3. How does temporal resolution change the relative value of relational and memory mechanisms?

The 5min and 1min regimes solve the same 30-minute lookback and 5min horizon task, while the 1sec regime uses a frequency-adapted two-minute lookback and two-minute horizon. This design allows the thesis to examine whether richer relation handling and recurrent memory become more useful as the observation frequency increases.

RQ4. Are the conclusions stable under deployment-oriented model states?

The thesis distinguishes between **last-fold-M**, the final walk-forward fold model used as the primary deployment-oriented reference, and **final-refit-M**, a model refit on a larger pre-holdout sample. This question asks whether the same model remains attractive when viewed through both states, and what this implies for realistic deployment interpretation.

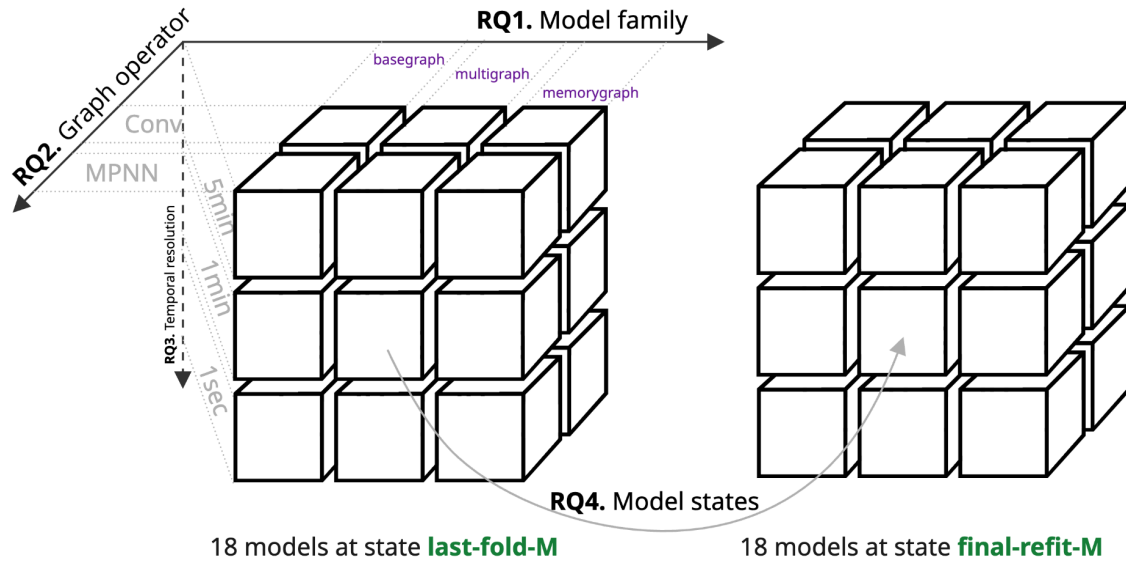


Figure 1.2 - Research questions and benchmark dimensions

Figure 1.2 shows that the main objects of the research are 36 models, which are built with an aim to answer research questions, assuming shared data, shared targets, shared validation and shared backtest (common controlled benchmark).

Table 1.1. Research questions, methods, and expected result types.

Research question	Applied research method	Expected result type
RQ1. Which graph family performs best?	Controlled benchmarking across basegraph , multigraph , and memorygraph under the same data, labels, splits, thresholds, and backtest.	Ranked model-family comparison on final-holdout economic and diagnostic metrics.
RQ2. How important is the Conv-versus-MPNN operator choice?	Within-family ablation-style comparison of Conv and MPNN graph operators across three frequencies.	Operator-level evidence showing whether performance changes are family- and frequency-dependent.
RQ3. How does temporal resolution affect relation and memory mechanisms?	Frequency-regime comparison between 5min, 1min, and frequency-adapted 1sec experiments.	Interpretation of how gross signal, turnover, and post-cost outcomes change with temporal resolution.
RQ4. Are conclusions stable between last-fold-M and final-refit-M ?	Deployment-state comparison using selected final-holdout benchmark results.	Stability assessment that separates chronological deployment evidence from larger-sample refit evidence.

1.5. Hypotheses

The empirical design tests five hypotheses.

H1. The one-minute regime should be the strongest shared-task benchmark.

Because the 1min data preserve more intra-horizon dynamics than 5min data while remaining less noisy than second-level data, the 1min regime is expected to be the strongest of the two strict shared-task regimes.

H2. Explicit multi-relation modelling should outperform the simpler baseline more clearly at finer resolutions.

The **multigraph** family is expected to benefit from preserving price-dependence, order-flow, and liquidity channels separately, especially when cross-asset dependencies evolve quickly (1min and 1sec regimes).

H3. Stateful memory should become more valuable as the market is observed more finely.

The **memorygraph** family is expected to be most useful at high frequency (1sec regime) because recurrent state can retain short-lived market information across contiguous observations.

H4. Conv and MPNN operators should not be uniformly dominant across families.

The operator comparison is expected to be family and frequency dependent. A Conv-style operator may be more stable when edge structure is already regularized, while an MPNN-style operator may be more useful when messages need richer source-destination-edge conditioning.

*H5. **last-fold-M** and **final-refit-M** should provide broadly consistent but not necessarily equivalent evidence.*

The broad family-level conclusion is expected to remain similar across both states **last-fold-M** and **final-refit-M**, while individual model profitability and diagnostic metrics may change when the larger pre-holdout sample is used for refitting. The hypothesis additionally treats the comparison as evidence about deployment sensitivity.

1.6. Thesis Contribution

The thesis contributes a controlled empirical comparison of graph-based market microstructure models across three temporal resolutions. This contribution provides evidence on three narrower issues:

1. whether a simpler single-graph baseline is sufficient under a common entry-model benchmark;
2. whether explicit multi-relation handling or stateful memory improves economic outcomes after costs;
3. why deployment-oriented model states, turnover, and cost drag must be included in the interpretation of high-frequency predictive models.

From a data science perspective, the thesis contributes as a controlled evaluation workflow for comparing models on noisy, non-stationary sequential data. The workflow combines graph-based representation learning, leakage-aware preprocessing, chronological validation, multi-task supervised learning, cost-sensitive post-processing, and metric selection. These elements are central to the thesis because the empirical conclusions depend on the full evaluation design: model architecture, data splitting, scaling, labelling, calibration, and final trading-oriented assessment.

The main empirical finding is conservative. Under the controlled benchmark, the tested GNN architectures behave differently across temporal resolutions, graph families, and deployment-oriented model states. In the **last-fold-M** setting, **base-gnn-conv** is the most stable configuration at both 5min and 1min. Richer graph mechanisms (**multigraph** and **memorygraph**) reveal useful diagnostic patterns, especially in gross signal extraction, ranking quality, turnover, and cost sensitivity at 1sec.

2. Literature Background

2.1. Market Microstructure and Limit Order Book Prediction

Market microstructure studies how trading rules, liquidity provision, order-flow, and the organization of the limit order book affect price formation [1], [2]. At short horizons, the visible book is informative because it contains the current distribution of buy and sell interest around the mid-price. However, short-horizon predictability is difficult to exploit. Return distributions are heavy-tailed, volatility clusters over time, and dependence structures change [3]. These features make financial forecasting different from many supervised-learning problems with stable sampling assumptions.

LOB modelling also creates an evaluation challenge. A direction classifier can appear useful under accuracy or area under the ROC¹ curve (AUC) while remaining economically weak if it triggers many low-margin trades. This gap is especially important in cryptocurrency markets. Empirical work on Bitcoin LOBs reports that crypto order books share some stylized facts with mature markets but can also be relatively shallow, with higher liquidity costs [4]. These conditions make transaction costs and turnover central to interpretation.

For this reason, this thesis evaluates models through a friction-aware entry benchmark rather than through classification metrics alone. Directional AUC (`dir_auc`) and trade AUC (`trade_auc`) are retained as diagnostics, but `net_pnl`, `gross_pnl`, and `n_trades` are required to interpret whether a predictive signal survives as an economic signal.

2.2. Deep Learning for Limit Order Books

Deep learning research on LOBs has shown that neural networks can learn representations from high-dimensional book states [5], [11]. DeepLOB is especially relevant because it combines convolutional components for local book structure with recurrent components for temporal dependence [7]. Large-scale order-flow studies also suggest that neural models can identify cross-instrument regularities in price formation [6]. These studies motivate representation learning in market microstructure, but they also highlight the need to separate predictive performance from economic performance.

Recent LOB research has broadened the model space. Arroyo et al. use convolutional-transformer survival models to estimate fill probabilities, connecting LOB representation learning to execution-aware decisions [8]. Jung and Lee study attention-based sequence-to-sequence forecasting of multi-level LOB states, emphasizing high dimensionality, irregular timing, and spatiotemporal dependencies [9]. Briola, Bartolucci, and Aste propose HLOB, which uses information-filtering graph structure to study information persistence across LOB levels [10]. These studies are not direct baselines for the present thesis, because the current benchmark is a cross-asset graph entry-model comparison rather than a full LOB reconstruction or execution-probability model. They are nevertheless relevant because they show that recent LOB research increasingly treats market microstructure as structured, temporal, and evaluation-sensitive data.

The present thesis differs from single-instrument LOB prediction studies by making cross-asset relational structure explicit. Instead of treating ETH only as an isolated time series, ADA and BTC are included as context nodes. The resulting graph is small, but it

¹ROC - Receiver-operating characteristic

allows the thesis to test whether graph modelling adds value once all families share the same target construction, validation protocol, and trading evaluation.

2.3. Graph Neural Networks and Message Passing

Graph neural networks (GNNs) provide a general framework for learning from entities connected by relations. Graph convolutional networks and graph attention networks show how node representations can be updated using neighbourhood information, while message-passing neural networks provide a flexible formulation in which messages depend on source nodes, destination nodes, and edge attributes [13], [14], [15]. These ideas are directly relevant to financial data because assets can be represented as nodes and cross-asset dependence measures as edges.

For a compact visual intuition, Figure 2.1 shows the layer-wise graph-convolution view. Each graph layer updates the embedding of a node by combining its previous representation with information propagated through the graph [20].

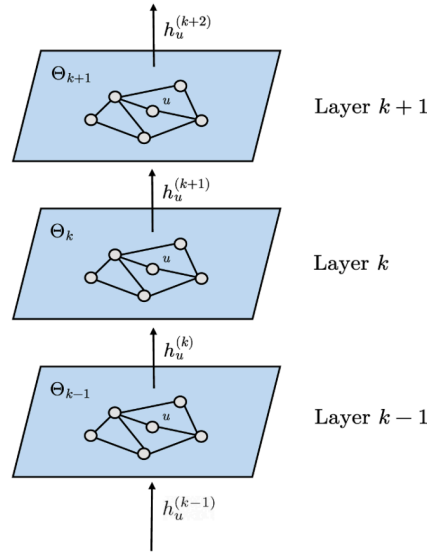


Figure 2.1 - Layer-wise graph-convolutional update of node embeddings.
(Adapted from [20], Fig. 8(b))

In this thesis, the Conv-versus-MPNN distinction is used as a controlled operator comparison. Conv means a convolution-style graph: layers apply weighted source-node projections with edge-conditioned shifts. MPNN means message-passing neural network: layers use richer gated messages that depend on source state, destination state, and edge state. The comparison therefore asks whether richer local message conditioning is economically useful under the same model family.

2.4. Temporal and Dynamic Graph Learning

Many real systems are not static graphs. Node states, edge states, and interaction patterns can evolve over time. Temporal graph networks and dynamic graph representation learning address this problem by combining graph operators with temporal encoders, memory modules, or event-driven updates [16], [17], [18]. Recent surveys reinforce that dynamic GNNs are designed for settings where topology or attributes change over time, and that open challenges include scalability, heterogeneous information, and memory-enhanced modelling

[19], [20]. This literature is relevant to market microstructure because cross-asset relations are unlikely to remain fixed across regimes, liquidity states, and trading intensity.

The three families in this thesis instantiate this idea at different levels of complexity:

- `basegraph` uses early relation fusion and a convolutional temporal backbone.
- `multigraph` preserves relation-specific graph pathways longer before fusing them.
- `memorygraph` uses recurrent node and edge memory inside a graph-processing loop.

The empirical question is not whether these mechanisms are theoretically plausible; it is whether they improve a controlled friction-aware benchmark after costs.

2.5. Financial Graph Learning

Financial applications of graph neural networks include stock relation modelling, portfolio prediction, risk propagation, fraud detection, and transaction-network analysis [21]. Financial graphs are often heterogeneous or time-varying because relations can be induced by sectors, ownership, supply chains, correlations, news, or market co-movements [21]. Recent multi-relational dynamic graph work is especially relevant because it treats financial relations as heterogeneous and temporally evolving rather than as a single fixed adjacency matrix [22].

This thesis adopts a microstructure version of the same general idea by constructing relation channels from price dependence, order-flow dependence, and liquidity dependence. The design is deliberately modest: it does not claim to solve dynamic financial graph learning in general. Its contribution is an internally controlled comparison on a specific crypto LOB dataset and a specific entry-model benchmark.

2.6. Evaluation, Backtesting, and Leakage Control

Financial machine learning requires evaluation methods that differ from many standard predictive-modelling workflows. Labels can depend on future price paths, adjacent samples can share overlapping horizons, and repeated experimentation can create backtest overfitting. The triple-barrier method and cross-validation framework are therefore central references for the target and validation design used in this thesis [12].

Transaction-cost-aware evaluation is also required. A high-frequency model may produce useful ranking statistics or positive gross PnL while failing after fees, spread, slippage, and turnover are considered. Execution-aware LOB research emphasizes that order placement and fill probabilities are themselves prediction problems, not details that can be ignored [8]. Algorithmic and high-frequency trading texts similarly treat costs, adverse selection, inventory risk, and execution design as central constraints on trading value [23]. The present thesis therefore keeps the backtest simple and common across models, but interprets `net_pnl`, `gross_pnl`, `pnl_per_trade`, `n_trades`, and `trade_rate` together.

The literature therefore motivates the empirical design that follows. Market microstructure explains why the prediction problem is noisy and friction-sensitive; deep LOB models motivate neural representation learning; GNN and temporal graph methods motivate cross-asset and memory-aware architectures; and financial machine learning evaluation literature motivates triple-barrier labels, walk-forward validation, and post-cost interpretation. The next chapter translates these ideas into the controlled data representation and evaluation protocol used in the thesis.

3. Data and Methodology

The core of the idea is the usage of financial-graph to extract and process more than one-asset information to apply it for trading, which is shown in Figure 3.1.

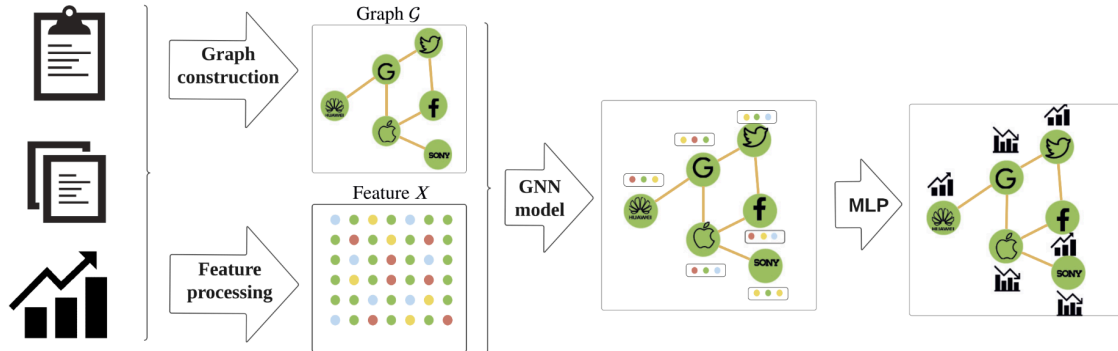


Figure 3.1 - General financial graph-learning pipeline. (Adapted from [21], Fig. 1)

In a financial GNN pipeline, market entities can be represented as vertices, interaction or dependence channels as directed edges, and observed market variables as node or edge features processed by a graph model [21]. As mentioned in paragraph 2, this idea is not new, but there is a lack of scientific papers, considering this approach for fine-grained time intervals (5min, 1min, 1sec) as considered in this thesis, while available papers interpret 1day or even wider intervals. In this thesis, the same idea is specialized to market microstructure: ADA, BTC, and ETH are the vertices, ETH is the target asset, and directed cross-asset edges carry rolling price-dependence, order-flow, and liquidity relations.

3.1. Data Source and Study Universe

The raw data source is the public Kaggle dataset *High-Frequency Crypto Limit Order Book Data*, which provides frequency-specific cryptocurrency limit order book snapshots for multiple assets, including ADA, BTC, and ETH, at 1sec, 1min, and 5min resolutions [24]. The data are distributed as order book snapshots organized by price level rather than as raw exchange message streams.

The present study uses a fixed three-node asset universe:

1. ADA
2. BTC
3. ETH

ETH is the target asset, selected after preliminary inspection of the available data. ADA and BTC provide relational market context. Because the source data are already distributed in frequency-specific tables, no bespoke reconstruction of the limit order book from raw order messages is required. The preprocessing task is instead to standardize timestamps, align assets on a common clock, and derive node and edge features from the available order book summaries.

The choice of these three assets is therefore methodological. ADA, BTC, and ETH are jointly available in the same public LOB dataset at 5min, 1min, and 1sec resolutions, which makes them suitable for a controlled graph benchmark. The graph representation is useful because

crypto-markets tend to show interconnectivity within major crypto assets, such as ETH, ADA and BTC. A larger universe would create a richer graph, but it would also introduce a separate data-engineering problem involving comparable LOB access, paid or exchange-specific API coverage, timestamp synchronization, missing levels, and venue-specific costs, which is out of scope of this financial-ML-oriented thesis. The corresponding external-validity limitation is discussed in [Section 6.5](#). Figure 3.2 gives a compact exploratory data analysis (EDA) view of this universe at the representative 1min resolution.

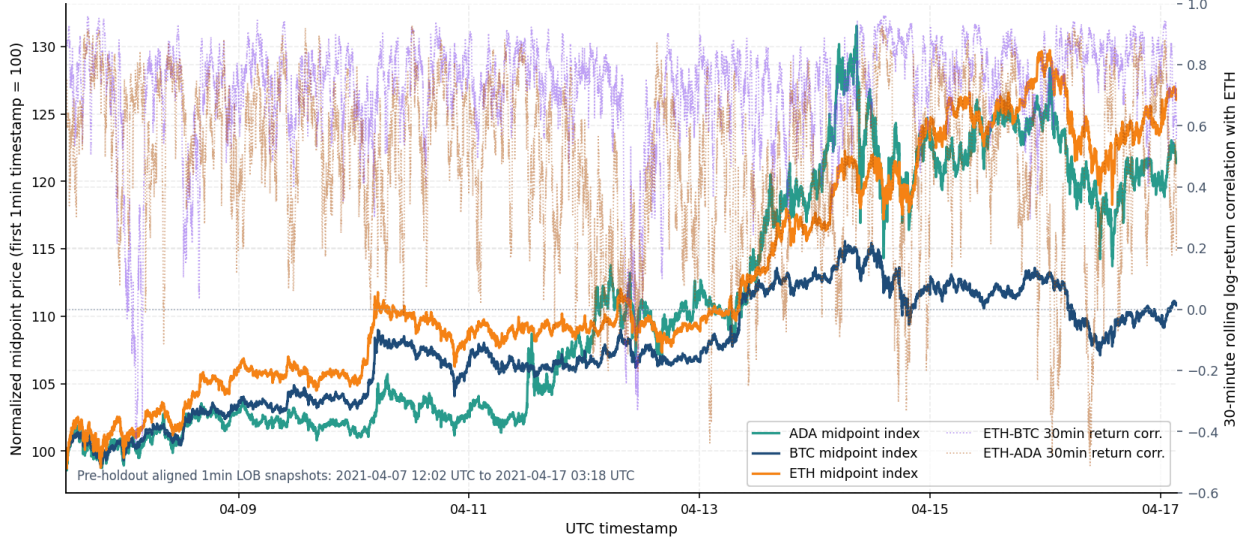


Figure 3.2 - Pre-holdout exploratory overview of the three-asset study universe. The left axis shows normalized midpoint prices; the right axis shows 30-minute rolling log-return correlations with the ETH target asset.

Midpoint prices are indexed to the first common timestamp on the left axis, while the right axis reports 30-minute rolling log-return correlations between ETH and the two context assets. Because the plot stops at the pre-holdout boundary, it motivates the study universe without using final-holdout information. The common price regime and time-varying target-centered correlations support treating ADA, BTC, and ETH as connected market nodes rather than as independent tabular inputs.

The local data files used in the pipelines contain midpoint price, spread, buy and sell flow summaries, and 15 bid-side and 15 ask-side depth values. These fields are the foundation for all node features and relation features used in the benchmark.

3.2. Graph Input Representation

All models use the same graph input representation within a frequency regime. The graph is a directed complete graph over the three assets with self-loops. The nodes are fixed, but node states and edge states vary over time.

Formally, each model receives:

1. a node sequence $X^{(n)} \in \mathbb{R}^{B \times L \times N \times F_n}$
2. a relation-aware edge sequence $X^{(e)} \in \mathbb{R}^{B \times L \times R \times E \times F_e}$

where

B is batch size,

L is the lookback length,

$N = 3$ is the number of assets,

$R = 3$ is the number of relation channels (price_dep, order_flow, liquidity),

$E = 9$ is the number of directed edges including self-loops ($3 \text{ assets} \times 3 \text{ out edges per asset}$),

$F_n = 15$ is the number of node features per asset, and

$F_e = 27$ is the number of edge features per relation channel ($3 \text{ lags} \times 3 \text{ windows} \times 3 \text{ statistics}$)

Sections 3.3 and 3.4 describe how the node features and relation-aware edge features are constructed from the raw limit order book fields. The overall graph representation is summarized in Figure 3.3.

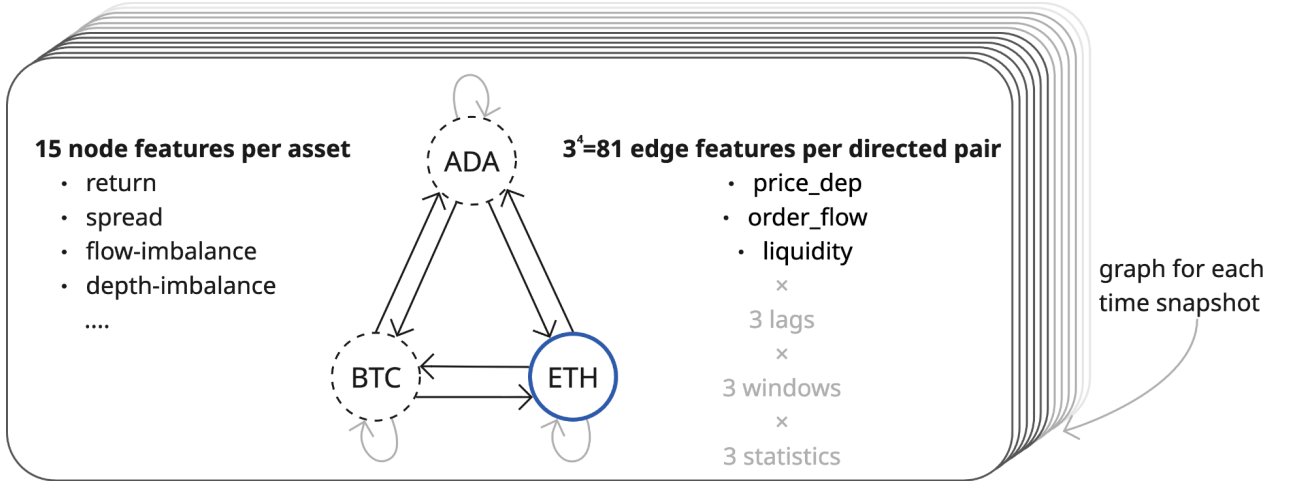


Figure 3.3 - Graph input representation

All families receive the same graph-structured input: a three-node directed complete graph with self-loops, ETH as target, dynamic node states, and three relation-aware edge channels.

3.3. Node Features

For each asset and each time step, the node feature block summarizes local price behavior, order-flow pressure, and depth structure. Let $m_{a,t}$ denote the midpoint price, $s_{a,t}$ the spread, $u_{a,t}$ and $v_{a,t}$ the buy and sell flow summaries, and $B_{a,t,k}$ and $A_{a,t,k}$ the bid-side and ask-side depth values at book level k , for asset a and time t . The benchmark uses 15 book levels, of which the first five are treated as near levels.

For compact notation, define near and far depth aggregates as

$$B_{a,t}^{\text{near}} = \sum_{k=0}^4 B_{a,t,k}, \quad A_{a,t}^{\text{near}} = \sum_{k=0}^4 A_{a,t,k}, \quad (1)$$

$$B_{a,t}^{\text{far}} = \sum_{k=5}^{14} B_{a,t,k}, \quad A_{a,t}^{\text{far}} = \sum_{k=5}^{14} A_{a,t,k}, \quad (2)$$

The implemented node tensor contains exactly $F_n = 15$ scalar features per asset. Table 3.1 lists the 15 node features in the same order as the feature-construction code.

In the formulas below, ϵ denotes a small numerical-stability constant, set to $\epsilon = 10^{-12}$ in the implementation. It is added to denominators to avoid division by zero when flow or depth values are zero. Since ϵ is negligible relative to observed price, flow, and depth magnitudes, it does not change the economic interpretation of the features; it only ensures that all feature computations are well defined.

Table 3.1. Implemented node features.

#	Node feature	Computation	Meaning
1	lr_1bar	$r_{a,t} = \log(m_{a,t}) - \log(m_{a,t-1}) \quad (3)$	One-bar local price movement.
2	rel_spread	$\tilde{s}_{a,t} = \frac{s_{a,t}}{m_{a,t} + \epsilon} \quad (4)$	Spread scaled by the asset price level.
3	log_buys	$\log(1 + u_{a,t}) \quad (5)$	Buy-side activity magnitude with logarithmic compression.
4	log_sells	$\log(1 + v_{a,t}) \quad (6)$	Sell-side activity magnitude with logarithmic compression.
5	flow_imbalance	$\frac{u_{a,t} - v_{a,t}}{u_{a,t} + v_{a,t} + \epsilon} \quad (7)$	Directional pressure between buy and sell flow.
6	depth_imbalance_total	$\frac{\sum_{k=0}^{14} B_{a,t,k} - \sum_{k=0}^{14} A_{a,t,k}}{\sum_{k=0}^{14} B_{a,t,k} + \sum_{k=0}^{14} A_{a,t,k} + \epsilon} \quad (8)$	Aggregate bid-versus-ask depth imbalance across the full book snapshot.
7	top_imbalance_0	$\frac{B_{a,t,i} - A_{a,t,i}}{B_{a,t,i} + A_{a,t,i} + \epsilon} \quad (9)$ $\forall i \in [0, 1, 2, 3, 4]$	Bid-ask imbalance at top-book level $\{i\}$.
8	top_imbalance_1		
9	top_imbalance_2		
10	top_imbalance_3		
11	top_imbalance_4		
12	bid_near_far_ratio	$\frac{B_{a,t}^{\text{near}}}{(B_{a,t}^{\text{far}} + \epsilon)} \quad (10)$	Concentration of bid liquidity near the best quotes relative to deeper levels.
13	ask_near_far_ratio	$\frac{A_{a,t}^{\text{near}}}{(A_{a,t}^{\text{far}} + \epsilon)} \quad (11)$	Concentration of ask liquidity near the best quotes relative to deeper levels.
14	depth_imbalance_near	$\frac{B^{\text{near}}_{a,t} - A^{\text{near}}_{a,t}}{B^{\text{near}}_{a,t} + A^{\text{near}}_{a,t} + \epsilon} \quad (12)$	Bid-versus-ask pressure close to the best quotes.
15	depth_imbalance_far	$\frac{B^{\text{far}}_{a,t} - A^{\text{far}}_{a,t}}{B^{\text{far}}_{a,t} + A^{\text{far}}_{a,t} + \epsilon} \quad (13)$	Bid-versus-ask pressure deeper in the book.

Thus, the node feature block contains one return feature, one spread feature, two log-flow features, one flow-imbalance feature, one total-depth feature, five top-level imbalance features, two near/far ratio features, and two near/far imbalance features.

This feature set is deliberately microstructure-oriented. It does not use external news, social media, or macroeconomic variables. That choice keeps the thesis focused on the information available inside the aligned cross-asset order book state.

3.4. Relation States and Edge Features

Edge features are constructed in two steps. First, the pipeline derives three asset-level relation states from the same order book quantities used in the node features. Second, for every ordered asset pair and every relation channel, it computes short-horizon rolling pairwise dependence features.

The first relation state is the price-dependence state:

$$q_{a,t}^{\text{price}} = r_{a,t} = \log(m_{a,t}) - \log(m_{a,t-1}) \quad (14)$$

This channel captures short-horizon cross-asset co-movement in returns. For instance, on the directed edge BTC-to-ETH, the corresponding features summarize how lagged BTC returns are associated with current ETH returns over the configured rolling windows.

The second relation state is the order-flow state. The implementation first computes the normalized buy-sell flow imbalance; this imbalance is then scaled by log turnover:

$$q_{a,t}^{\text{flow}} = \text{flow_imbalance}_{a,t} \times \log(1 + u_{a,t} + v_{a,t}) = \frac{u_{a,t} - v_{a,t}}{u_{a,t} + v_{a,t} + \epsilon} \times \log(1 + u_{a,t} + v_{a,t}) \quad (15)$$

It is a signed buy-sell flow state built from the available snapshot-level flow fields: the normalized term captures direction, while the log-turnover multiplier increases the magnitude when the directional imbalance occurs during more active periods.

The third relation state is the liquidity state. It combines spread, depth imbalance, and the near/far shape of the book. Let $\rho_{a,t}^B$ denote the bid near/far ratio and $\rho_{a,t}^A$ denote the ask near/far ratio. The near/far book-shape term is

$$g_{a,t} = \tanh \left(\log \frac{\rho_{a,t}^B + 1}{\rho_{a,t}^A + 1} \right) \quad (16)$$

The implemented liquidity relation state is

$$q_{a,t}^{\text{liq}} = -\log(1 + 10^4 \tilde{s}_{a,t}) + 0.50 I_{a,t}^{\text{total}} + 0.25 I_{a,t}^{\text{near}} + 0.25 g_{a,t} \quad (17)$$

where $\tilde{s}_{a,t}$ is relative spread, $I_{a,t}^{\text{total}}$ is total depth imbalance, and $I_{a,t}^{\text{near}}$ is near-depth imbalance. The negative spread term lowers the state when trading conditions are wider and

therefore less liquid, while the imbalance and shape terms retain information about the distribution of available depth across the two sides of the book.

Once the three relation-state series have been constructed, the edge features are computed in the same way for every relation channel. For each channel c , each directed edge $i \rightarrow j$, each lag $h \in 0, 1, 2$, and each frequency-specific rolling window $w \in W_f$ (defined in the code), the implementation forms three rolling statistics.

The first statistic is rolling correlation, with Fisher-z transformation enabled in the final benchmark configurations:

$$\text{corr}_{i \rightarrow j, t}^{c, h, w} = z \left(\text{Corr}_w \left(q_{i, t-h}^c, q_{j, t}^c \right) \right) \quad (18)$$

Where $z(x) = 0.5 \log \frac{1+x}{1-x}$ (Fisher-z transformed)

The second statistic is a beta-style dependence coefficient:

$$\beta_{i \rightarrow j, t}^{c, h, w} = \frac{\text{Cov}_w \left(q_{i, t-h}^c, q_{j, t}^c \right)}{\text{Var}_w \left(q_{i, t-h}^c \right) + \epsilon} \quad (19)$$

The third statistic is the rolling mean product:

$$\mu_{i \rightarrow j, t}^{c, h, w} = \text{Mean}_w \left(q_{i, t-h}^c q_{j, t}^c \right) \quad (20)$$

Together, these statistics capture complementary forms of short-horizon pairwise dependence: normalized co-movement, directed beta-style sensitivity, and average signed interaction. Since the benchmark uses three relation channels, three lags, three windows, and three rolling statistics, each directed asset pair receives $3 \times 3 \times 3 \times 3 = 81$ handcrafted relation-derived edge features. In tensor form, these are stored as $C = 3$ relation channels with $F_e = 27$ edge features per channel. The handcrafted relation-derived features are fused with learned source-destination node interactions. These learned interactions are part of the model input pipeline, but they are not separate precomputed edge-feature columns.

This construction keeps the comparison controlled. All three model families receive the same relation states and the same handcrafted edge-feature statistics; they differ in how they process, aggregate, and fuse this information, not in whether one family is given richer input data than another. The shortest rolling windows should therefore be interpreted as local co-movement proxies used for controlled benchmarking, rather than as stable long-horizon estimates of market dependence.

3.5. Scaling and Leakage Control

Node and edge tensors are robustly scaled on training data only. The implementation uses 'RobustScaler' with centering and scaling based on the 5th and 95th percentiles. For node features, the scaler is fitted on the training portion of the node tensor after flattening the time and asset dimensions. For edge features, a separate scaler is fitted for each relation channel after flattening the time and directed-edge dimensions.

This fold-specific scaling prevents train-test leakage because validation and holdout observations are transformed using statistics estimated only from the corresponding training

interval. It also reduces the influence of extreme observations, which is important for LOB-derived features with heavy tails and occasional very large depth or flow values. Because the same scaling procedure is used for all graph families, feature preprocessing does not favor any architecture.

Leakage control is especially important in this thesis because the target labels are constructed from future ETH midpoint paths. The validation design therefore avoids random splits and treats chronological separation, fold-specific preprocessing, and purge gaps as part of the experimental method rather than as implementation details. This follows the financial machine learning argument that conventional cross-validation can be misleading when labels overlap in time or when repeated strategy selection creates overfitting risk [12].

3.6. Frequency-Specific Experimental Regimes

The experimental design contains eighteen primary runs: six model variants for each of the three frequency regimes.

The 5min and 1min regimes solve the same clock-time task:

1. lookback window = 30 minutes.
2. forecast horizon = 5 minutes.

This corresponds to six lookback bars and one horizon bar at 5min, and 30 lookback bars and five horizon bars at 1min.

The 1sec regime uses a frequency-adapted task:

1. lookback window = 2 minutes = 120 bars.
2. forecast horizon = 2 minutes = 120 bars.

The 1sec working sample is restricted to the interval from 50% to 90% of the full second-level series. This keeps training computationally feasible while preserving a late-period high-frequency comparison. The final holdout fraction is increased to align the 1sec blind evaluation interval as closely as possible with the final holdout interval used in the slower-frequency experiments. Table 3.2 summarizes these frequency-specific settings.

Table 3.2. Frequency-specific experimental regimes and validation settings.

Frequency	Working data slice	Final holdout fraction	Lookback	Horizon	CV folds
5min	0.0-0.9 of the full series	0.10	30 min = 6 bars	5 min = 1 bar	4
1min	0.0-0.9 of the full series	0.10	30 min = 30 bars	5 min = 5 bars	4
1sec	0.5-0.9 of the full series	0.225	2 min = 120 bars	2 min = 120 bars	2

The frequency-specific sample design is shown schematically in Figure 3.4.

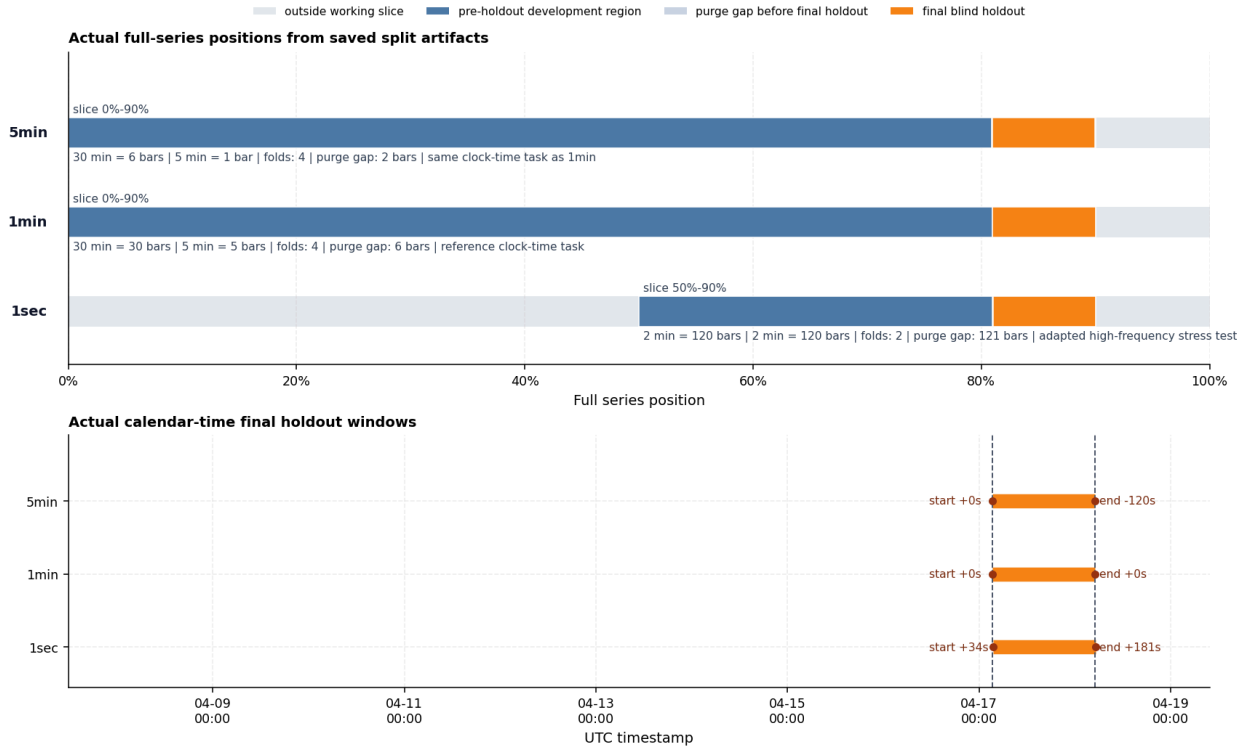


Figure 3.4 - Split design and Final Holdout difference

Figure 3.4 shows the actual usage of the available data. First of all the unseen final holdout orange-coloured regions are separated for every time frequency. Among frequencies these regions are aligned almost perfectly (1sec holdout differs unimportantly). This final holdout is crucial for testing of all the models. Secondly, the pre-holdout data (blue regions), which is the main data to train and evaluate all models. Thirdly, the last chronological part is the grey-colored region, which was not used in this thesis completely. This “out of scope” slice can be used for further development or additional testing of the models.

The 5min and 1min regimes share the same broad working-sample logic and late final holdout, so their results form the strict shared-task benchmark. The 1sec regime is internally consistent, but its restricted working slice (40% of entire dataset) and enlarged holdout show why it should be interpreted as a frequency-adapted high-frequency stress test rather than as a perfectly symmetric continuation of the 30-minute lookback and 5min horizon task. Appendix A provides a compact reference version of these frequency and model-configuration choices, while Appendix B records the run-level evidence that the final-holdout intervals are aligned as intended across frequencies.

3.7. Target Construction and Shared Learning Objective

All model families are trained under the same multi-task triple-barrier framework. The triple-barrier approach labels an observation by following the future price path until an upper barrier, lower barrier, or vertical time barrier is reached [12].

The barrier system is volatility-scaled (over a 30-bar rolling window). In the benchmark configuration, the upper and lower barriers start from 8 basis points, are rescaled using rolling volatility estimated over a 30-bar lookback, are multiplied by 1.8, and are clipped to the

interval from 4 to 30 basis points. The vertical barrier is set equal to the prediction horizon (5 minutes). Figure 3.5 summarizes the triple-barrier target construction used in the thesis.

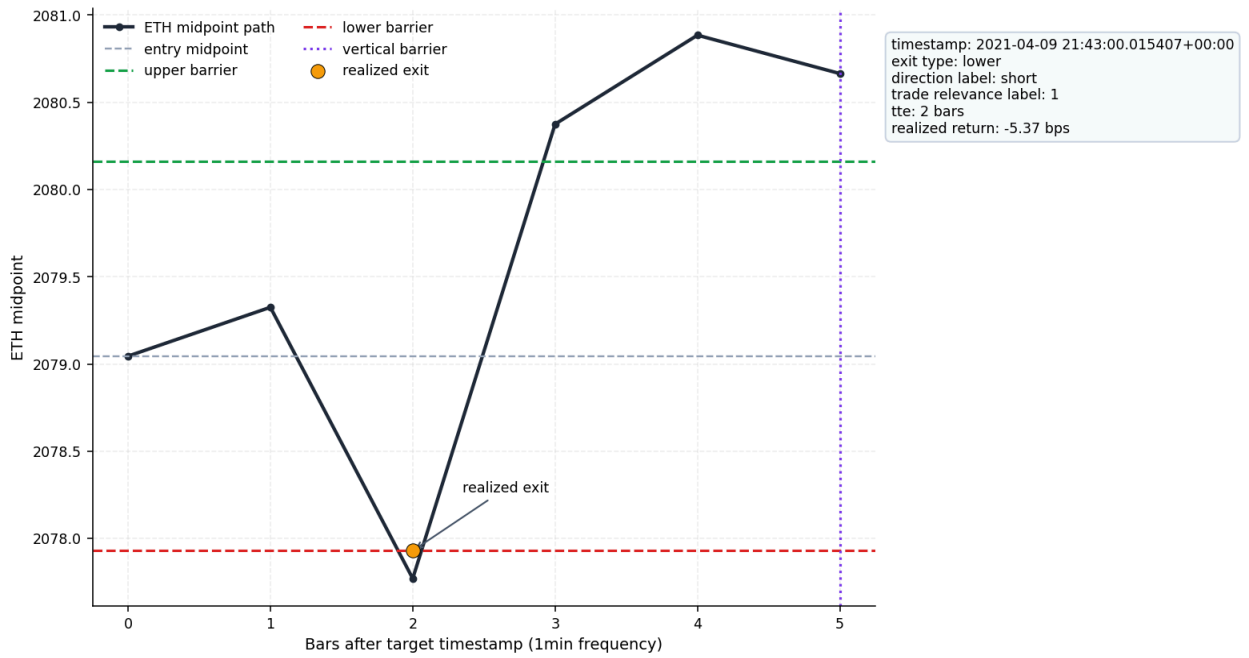


Figure 3.5 - Triple barrier construction

Figure 3.5 highlights that the label is path-dependent: the realized outcome is determined by the first touched barrier or by the vertical timeout (not always by the endpoint return at a fixed horizon). This is why the same construction can provide both an economic trade label and additional information about direction, exit type, and time to exit.

In this thesis, for each valid timestamp t , the future path of the ETH midpoint is followed until one of three mutually exclusive events occurs:

1. the upper barrier is touched (green dashed line)
2. the lower barrier is touched (red dashed line)
3. the vertical barrier is reached (blue dotted line)

From this future path, the pipeline constructs a common target set:

1. trade relevance label (trade_logit)
2. direction label (dir_logit)
3. realized return (return_pred)
4. exit-type label (exit_type_logit)
5. time-to-exit label (tte_pred)

The trade label is meta-labeled and depends on whether the future move remains economically meaningful after a friction-aware threshold is applied. Direction labels are masked when timeout outcomes are configured as uninformative for directional supervision. This design uses the triple-barrier framework as a labelling device, not as evidence that a trading strategy is economically complete.

The multi-task objective combines trade classification, direction classification, return regression, utility-based supervision, exit-type classification, and time-to-exit regression. In the benchmark configuration, the loss weights are:

1. $\text{loss_w_trade} = 0.35$
2. $\text{loss_w_dir} = 0.65$
3. $\text{loss_w_ret} = 0.15$
4. $\text{loss_w_utility} = 0.85$
5. $\text{loss_w_exit_type} = 0.05$
6. $\text{loss_w_tte} = 0.03$

These values are the default loss weights used as the main benchmark configuration. A small number of resolved runs use minor run-level deviations while preserving the same output heads and loss components. The two 1min `memorygraph` runs use a slightly stronger trade and utility emphasis: $\text{loss_w_trade} = 0.45$, $\text{loss_w_dir} = 0.55$, $\text{loss_w_ret} = 0.10$, and $\text{loss_w_utility} = 0.95$, while the four 1sec `multigraph` and `memorygraph` runs use $\text{loss_w_exit_type} = 0.01$ and $\text{loss_w_tte} = 0.05$. These deviations are treated as configuration-level robustness details rather than as separate benchmark dimensions.

The target construction, output heads, final holdout logic, and event-based backtest remain common across the benchmark, so the models still differ primarily in how they encode temporal and graph structure, not in the economic target or final backtest used to evaluate them.

3.8. Common Entry-Model Backtest

The trading evaluation is formulated as a common entry-model benchmark:

1. the trade head determines whether a trade candidate is active.
2. the direction head determines whether the candidate becomes a long or short position.
3. the exit is generated by the same realized event rule for all families.

Exit-type and time-to-exit heads are retained as additional learning targets and diagnostics, but they do not define a family-specific trade-closing policy in the main benchmark. This choice is especially important for `memorygraph`, because a stateful architecture could otherwise be evaluated under a different execution policy from the other families. The common entry-model benchmark improves internal validity by holding execution logic fixed.

The trading evaluation uses a sequential non-overlapping event-based backtest. Once a position is opened, no new position can be opened until the current one is closed. This makes turnover interpretable and avoids overlapping position exposure. The design is intentionally simpler than an execution simulator. Execution-aware LOB research shows that fill probabilities, passive-versus-aggressive order placement, and queue dynamics can materially affect realized trading value [8]. Algorithmic trading literature similarly treats transaction costs, market impact, and adverse selection as core constraints [23]. The present benchmark therefore uses a transparent cost proxy and interprets the result as an entry-model comparison, not as a live execution claim.

Trade and direction thresholds are selected on validation data only from finite threshold grids, subject to minimum-trade and minimum-coverage constraints. The selected thresholds are then held fixed for final-holdout evaluation.

For trade i , gross PnL is computed as:

$$gross_pnl_i = s_i \cdot r_i, \quad (21)$$

where $s_i \in \{-1, +1\}$ is the trade side and r_i is the realized log return up to the realized event exit. Net PnL is:

$$net_pnl_i = gross_pnl_i - c_{rt} \quad (22)$$

where the round-trip transaction-cost proxy is:

$$c_{rt} = cost_bps_unit = 3 \times 10^{-4} \quad (23)$$

The cost method is rather simple, but it is sufficient for a controlled friction-aware benchmark. All model families are evaluated under the same entry-model backtest: trade activation, direction selection, realized-event exit, and a shared transaction-cost proxy that converts gross PnL to net PnL.

3.9. Validation Design and Deployment-Oriented Model States

The experiments use chronological validation. Each working sample is divided into a pre-holdout region used for model development and a later final holdout region used only for blind final evaluation. Within the pre-holdout region, each walk-forward fold follows the same structure:

1. training window
2. purge gap
3. validation window
4. purge gap
5. chronological test window

The purge gaps are necessary because triple-barrier labels depend on future price paths. Adjacent observations can therefore have overlapping label windows, which would create leakage if neighbouring samples were placed directly on different sides of a split boundary. This design follows the financial machine learning recommendation that path-dependent labels require time-aware splitting and purging rather than ordinary random cross-validation [12]. It also reduces, but does not eliminate, model-selection and backtest-overfitting risk. Figure 3.6 shows the actual split geometry for representative 1min and 1sec runs on a shared calendar-time axis.



Figure 3.6 - Chronological fold design (chronological walk-forward folds)

The 1min panel starts near the beginning of the available working sample, while the 1sec panel starts later because the 1sec experiment uses a later working slice of the original second-level series. This makes the final holdout intervals comparable in calendar time even though the two frequency regimes use different working-sample starts and different purge gap lengths.

The final chronological walk-forward fold is especially important for the main benchmark. In Figure 3.6, its internal fold test window is followed by the orange final-holdout block. This indicates that the model obtained from the last walk-forward fold, denoted *last-fold-M*, is the primary deployment-oriented reference and is evaluated on the later blind holdout. The separate final refit row shows the larger pre-holdout refit setup used for *final-refit-M*, where training uses the largest available pre-holdout sample before the same blind holdout interval.

The study therefore distinguishes two model states:

1. *last-fold-M*, the model from the last chronological walk-forward fold and the primary deployment-oriented reference.
2. *final-refit-M*, the model refit on the largest possible pre-holdout sample and used as a larger-sample comparison rather than a replacement deployment state.

The main thesis benchmark uses *last-fold-M*. This state is *the most deployment-relevant reference* because it approximates a model selected from the most recent chronological validation cycle before the final refit. The *final-refit-M* state adds a useful larger-sample comparison, but it cannot replace *last-fold-M*: refitting changes the training sample, may change the score-to-trade conversion, so it is interpreted as a robustness comparison rather than as the primary deployment state.

3.10. Metrics

The main empirical metrics are:

1. *gross_pnl*, the sum of pre-cost directional trade returns
2. *net_pnl*, the sum of post-cost trade returns

3. `pnl_per_trade`, the average post-cost trade return
4. `n_trades`, the number of executed trades
5. `trade_rate`, the fraction of eligible events that become executed trades
6. `sign_accuracy`, the fraction of trades for which the predicted side matches the realized event direction
7. `win_rate`, the fraction of executed trades with positive net outcome
8. `sharpe_like`, a scale-free diagnostic of mean trade return relative to trade-return dispersion
9. `dir_auc`, the AUC of the direction head
10. `trade_auc`, the AUC of the trade head
11. `RMSE`, the return-regression error diagnostic

The primary economic metric is `net_pnl`. The `gross_pnl` metric separates raw signal extraction from the effect of transaction costs, while `n_trades` measures the turnover required to obtain that result. Together with `pnl_per_trade`, and `trade_rate` metrics we can analyze whether the result is supported by selective trading or by high turnover. The benchmark tables report these economic quantities together with `dir_auc` and `trade_auc` because the direction and trade heads are the two decision-critical heads of the entry model: they determine the predicted side and whether a trade is activated. Their AUC values therefore evaluate the core predictive components that feed the trading rule. This role is distinct from the operational diagnostics in Appendix C, including `pnl_per_trade`, `trade_rate`, `sign_accuracy`, `win_rate`, `sharpe_like`, and `RMSE`, which explain how a signal is translated into realized trading performance. Appendix A lists the shared target, loss, and cost configuration behind these metrics so that the metric interpretation can be traced back to the common benchmark setup.

3.11. Fair-Comparison Principle

At the intended benchmark-design level within each frequency regime, the primary architectural comparison varies two aspects:

1. the model family (`basegraph`, `multigraph`, `memorygraph`)
2. the local graph operator (Conv or MPNN)

The following elements are held fixed within a regime:

1. asset universe and target asset
2. node-feature construction
3. relation-state construction
4. edge-feature construction
5. label construction
6. multi-task output interface
7. thresholding logic
8. event-based backtest
9. split protocol
10. final holdout interval

Apart from the minor resolved-run loss-weight deviations noted in Section 3.7, these controls are the methodological basis for treating the benchmark as an architecture comparison rather than as a comparison of unrelated trading systems.

4. Model Families

4.1. Shared Architectural Conventions

All three model families operate on the same node and edge tensors and produce the same multi-task outputs. They also share a hybrid edge-fusion mechanism that augments handcrafted relation features with learnable pairwise node interactions. At the family level, the main architectural difference is how long relation information is kept separate during graph processing: **basegraph** fuses relation channels before message passing, **multigraph** processes relation channels separately before fusion, and **memorygraph** carries relation-aware node and edge states through a recurrent memory loop.

An additional, orthogonal comparison is the local graph operator. Each family is evaluated with both a Conv-style and an MPNN-style operator, so the benchmark separates the broader model family construction from the local message-update rule.

Table 4.1. Controlled architectural comparison across the three model families.

Design axis	basegraph	multigraph	memorygraph
Relation handling	Early fusion of relation channels	Separate relation pathways before fusion	Relation-aware recurrent state
Graph blocks	Single graph operator block	One graph block per relation	Graph operator inside a memory loop
Temporal mechanism	Causal convolutional encoders	Causal convolutional encoders	Recurrent node and edge memory across contiguous chunks
Statefulness	Window-based, no persistent memory	Window-based, no persistent memory	Persistent node and edge memory carried across chunks
Main architectural hypothesis	A fused relation representation is sufficient	Preserving relation channels improves graph updates	Recurrent state captures short-lived high-frequency structure

The three families therefore differ not by their input data, target construction, output heads, or evaluation protocol, but by the point at which relation information is fused and by whether temporal information is represented through convolutional windows or recurrent state. Chapter 4 should therefore be read as a controlled architectural ablation rather than as a comparison between models trained under different information sets. The architectural comparison is summarized in Figure 4.1.

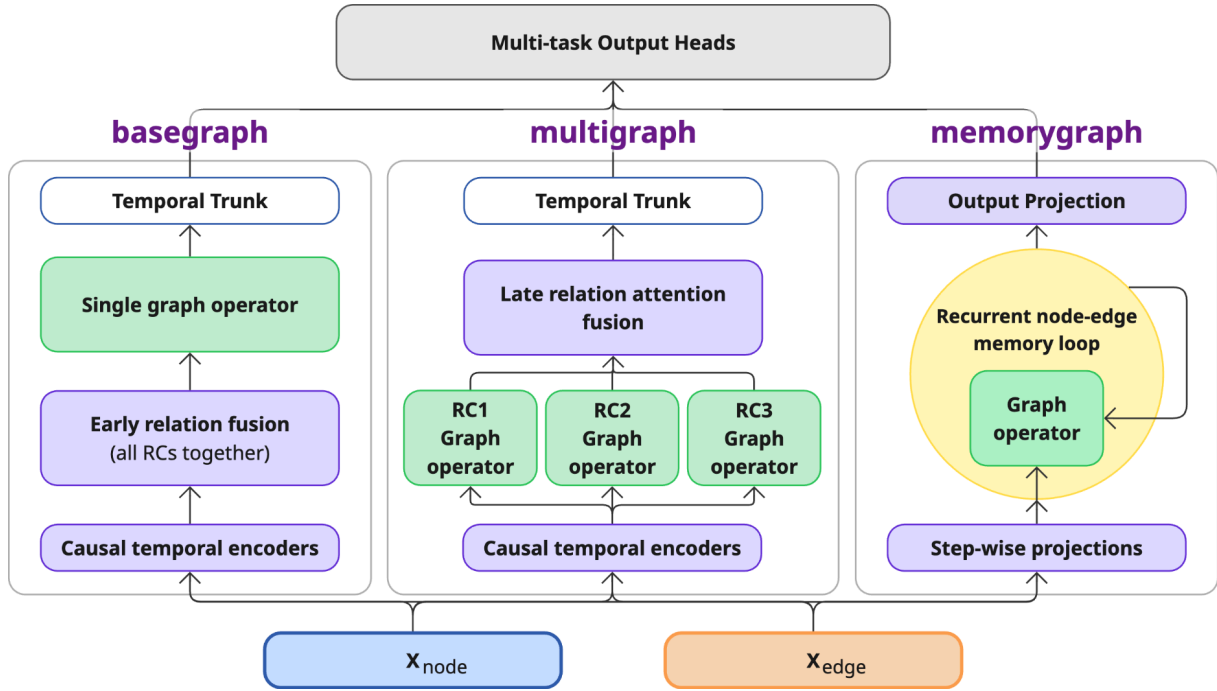


Figure 4.1 — Comparison of the Architectures

The family comparison is controlled because the models share the same data, target construction, and output heads. The family-level contrast concerns relation fusion and temporal state representation, while the Conv-versus-MPNN choice is tested as a separate within-family operator axis.

The two local graph operator types summary:

The Conv-style operator applies a weighted source-node projection plus an edge-conditioned shift term;

The MPNN-style operator computes gated messages conditioned on source node state, destination node state, and edge state;

This makes the MPNN operator more expressive, but not automatically more profitable.

4.2. The basegraph Family

The **basegraph** family is the single-graph baseline. It is evaluated through two adaptive operators, corresponding to the benchmark variants **base-gnn-conv** and **base-gnn-mpnn**:

1. `adaptive_conv`
2. `adaptive_mpnn`

Architecturally, **basegraph** tests whether the three relation channels can be compressed into a single edge representation before graph message passing without losing economically useful information.

The temporal component is fully convolutional. Node inputs are projected into hidden space, augmented with learned asset embeddings, and processed by dilated causal residual convolution blocks. Edge inputs are processed by a separate temporal edge encoder. After graph processing and readout, the target-centered sequence is passed through a second causal temporal trunk.

The following block gives a deliberately high-level, Python-like sketch of the `basegraph` computation.

```
def basegraph_forward(X_node, X_edge):
    node_seq = NodeTemporalEncoder(X_node)
    edge_seq = EdgeTemporalEncoder(X_edge)

    relation_edges = HybridEdgeFeatureFusion(node_seq, edge_seq)
    fused_edges = EdgeRelationFusion(relation_edges)
    # early relation fusion
    graph_node_seq = SingleGraphOperatorBlock(node_seq, fused_edges)
    readout_seq = GraphReadout(graph_node_seq, target_node="ETH")
    shared_state = TargetTemporalTrunk(readout_seq)

    return PredictionHeads(shared_state)
```

The decisive operation is `EdgeRelationFusion`: relation-aware edge features are collapsed into a single edge representation before graph message passing. A single graph operator block is then applied using adaptive adjacency. Early relation fusion therefore happens before graph message passing, and after that fusion the model has only one graph block. This makes `basegraph` the clean single-graph baseline of the benchmark.

The readout concatenates the target-node representation with global graph context, including mean and max pooling. The resulting target-centered representation is mapped to the shared multi-task prediction heads. Figure 4.2 visualizes this early-fusion single-graph design.

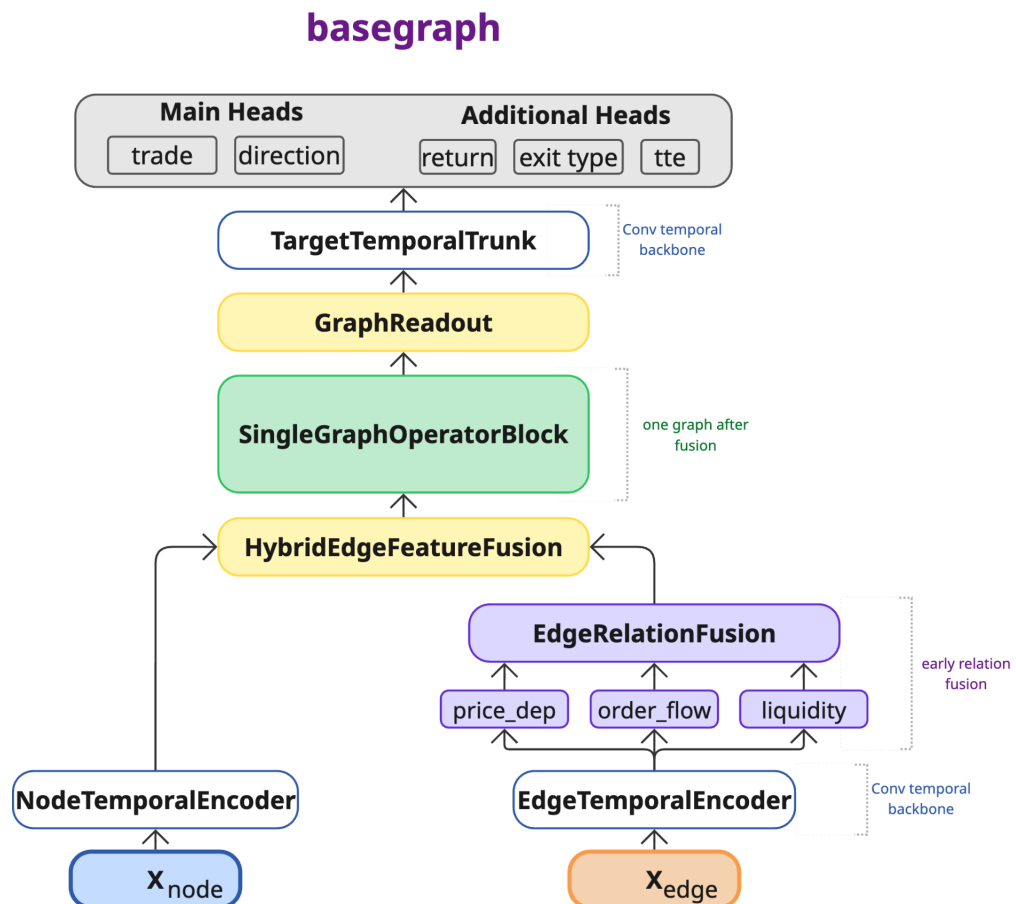


Figure 4.2. Detailed architecture of the `basegraph` family.

Therefore, `basegraph` compresses relation channels *before* graph message passing, then runs one graph block over the fused edge representation.

4.3. The multigraph Family

The `multigraph` family extends the baseline by preserving relation channels deeper into the graph-processing stage. It is evaluated in two matched variants, corresponding to `multi-gnn-conv` and `multi-gnn-mpnn`:

1. `dynamic_rel_conv`
2. `dynamic_edge_mpnn`

The temporal component is structurally similar to `basegraph`: node and edge histories are encoded with dilated causal convolution blocks, and the target readout is processed by a causal temporal trunk. The difference is in graph processing. Instead of collapsing the relation axis before message passing, the model constructs a separate relation graph block for each relation channel.

The following high-level pseudocode (architectural sketch) shows also a relation loop.

```
def multigraph_forward(X_node, X_edge):
    node_seq = NodeTemporalEncoder(X_node)
    edge_seq = EdgeTemporalEncoder(X_edge)

    relation_edges = HybridEdgeFeatureFusion(node_seq, edge_seq)

    relation_node_states = []
    for relation in ["price_dep", "order_flow", "liquidity"]:
        relation_node_states.append(
            RelationGraphBlock[relation](node_seq, relation_edges[relation])
        )

    graph_node_seq = RelationAttentionFusion(relation_node_states)
    # late fusion
    readout_seq = GraphReadout(graph_node_seq, target_node="ETH")
    shared_state = TargetTemporalTrunk(readout_seq)

    return PredictionHeads(shared_state)
```

For each relation, the Conv variant computes dynamic edge scores and applies normalized source-node projections and edge-conditioned shifts. The MPNN variant uses gated messages conditioned jointly on source state, destination state, and edge state. After relation-specific processing, the model applies learned relation attention fusion. The key architectural contrast with `basegraph` is therefore that `multigraph` delays relation fusion until after message passing. Price-dependence, order-flow, and liquidity induce separate node updates before learned relation attention combines them.

The central design question for `multigraph` is not simply whether a more complex model helps. It is whether relation semantics should remain separated during message passing, so

that price-dependence, order-flow, and liquidity can shape node updates differently before being merged into a shared representation. Figure 4.3 visualizes this late-fusion design.

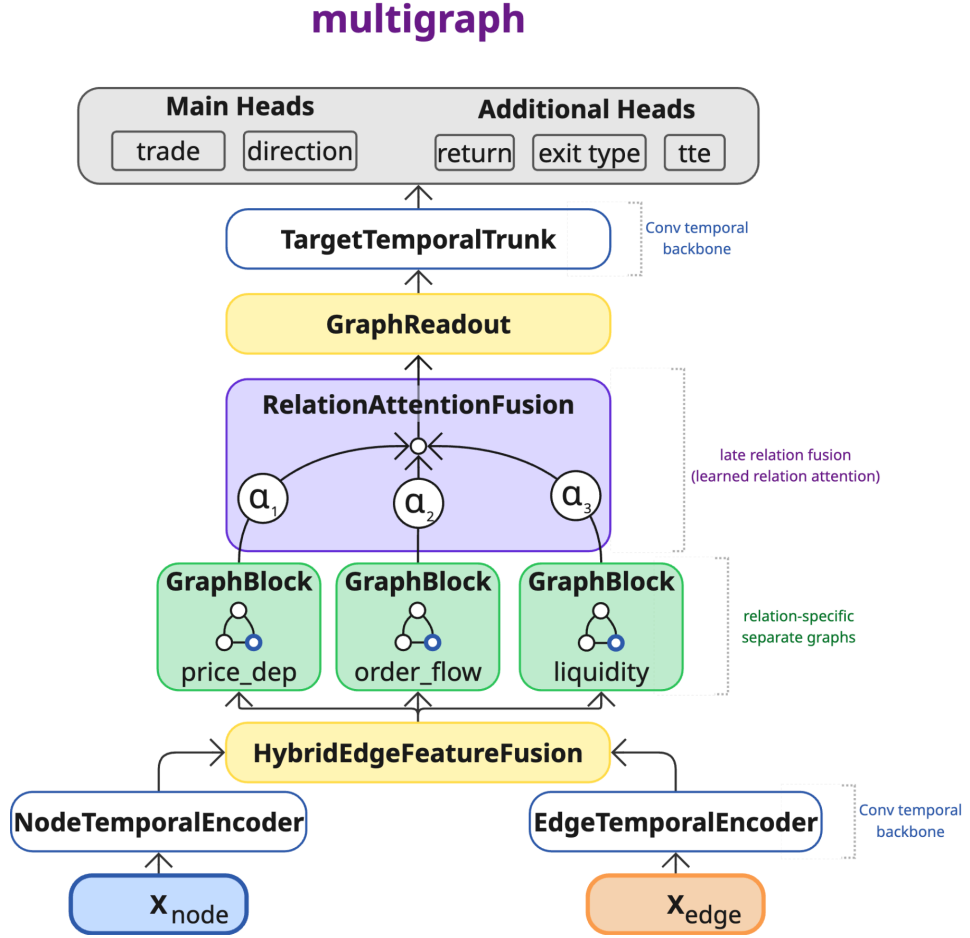


Figure 4.3. Detailed architecture of the multigraph family.

Therefore, **multigraph** preserves relation-specific semantics through message passing and only fuses them *after* relation-specific graph updates.

4.4. The memorygraph Family

The **memorygraph** family is the most distinct architecture in the study. It is evaluated with two variants, **memory-gnn-conv** and **memory-gnn-mpnn**:

1. conv
2. mpnn

Unlike **basegraph** and **multigraph**, it does not rely on a deep causal-convolutional temporal encoder. Instead, it uses stateful recurrent memory. Raw node and edge inputs are first projected at each time step. A MemoryAugmentedGraphBlock then maintains node memory and relation-specific edge memory across contiguous chunks.

The following pseudocode is a very high-level representation of the recurrent logic. It complements Figure 4.4 by making the update order explicit: current inputs are first enriched

with stored node and edge memory, edge memory is updated, relation-specific graph interaction is applied, and node memory is then updated.

```
def memorygraph_recurrent_forward(X_node_chunk, X_edge_chunk, state):
    node_memory, edge_memory = state
    fused_node_steps = []

    for X_node_t, X_edge_t in contiguous_chunk(X_node_chunk, X_edge_chunk):
        node_t = NodeStepProjector(X_node_t)
        edge_t = EdgeStepProjector(X_edge_t)
        relation_edges_t = HybridEdgeFeatureFusion(node_t, edge_t)

        relation_node_input = enrich_with_node_memory(node_t, node_memory)
        relation_edge_input = enrich_with_edge_memory(relation_edges_t,
        edge_memory)

        edge_memory, relation_edge_state = EdgeMemoryUpdater(
            relation_node_input, relation_edge_input, edge_memory
        )
        relation_node_states = MemoryOperatorBlock(
            relation_node_input, relation_edge_state
        )
        node_memory, fused_node_t = NodeMemoryUpdater(
            relation_node_states, edge_memory, node_memory
        )
        fused_node_steps.append(fused_node_t)

    fused_node_seq = stack(fused_node_steps)
    readout_seq = GraphReadout(fused_node_seq, target_node="ETH")
    shared_state = OutputProjection(readout_seq)

    return PredictionHeads(shared_state), [node_memory, edge_memory]
```

The edge memory update uses recurrent cells conditioned on current edge state, source-node state, destination-node state, and pairwise node interactions. The node memory update aggregates relation-specific edge-memory context to nodes, fuses relation-specific node and edge contexts, and updates node memory with another recurrent cell. Training uses contiguous stateful chunks with truncated backpropagation through time. In `memorygraph`, temporal modelling is therefore not primarily a convolution over a fixed lookback window. Temporal information is stored in recurrent node and relation-specific edge memories that are updated step by step across contiguous chunks.

Inside each recurrent step, the graph operator is either Conv-style or MPNN-style. The key difference from the other families is that graph interaction occurs inside a recurrent memory loop. The operator acts on state-enriched representations rather than on a fully pre-encoded temporal sequence. This makes `memorygraph` qualitatively different from the convolutional window-based logic of `basegraph` and `multigraph`, even when the same local Conv-versus-MPNN comparison is preserved.

This gives `memorygraph` a qualitatively different inductive bias:

1. `basegraph` uses early relation fusion and convolutional temporal modelling.
2. `multigraph` uses late relation fusion and convolutional temporal modelling.
3. `memorygraph` uses relation-aware recurrent state and stateful graph updates.

Figure 4.4 highlights the recurrent memory mechanism that differentiates **memorygraph** from the convolutional temporal families.

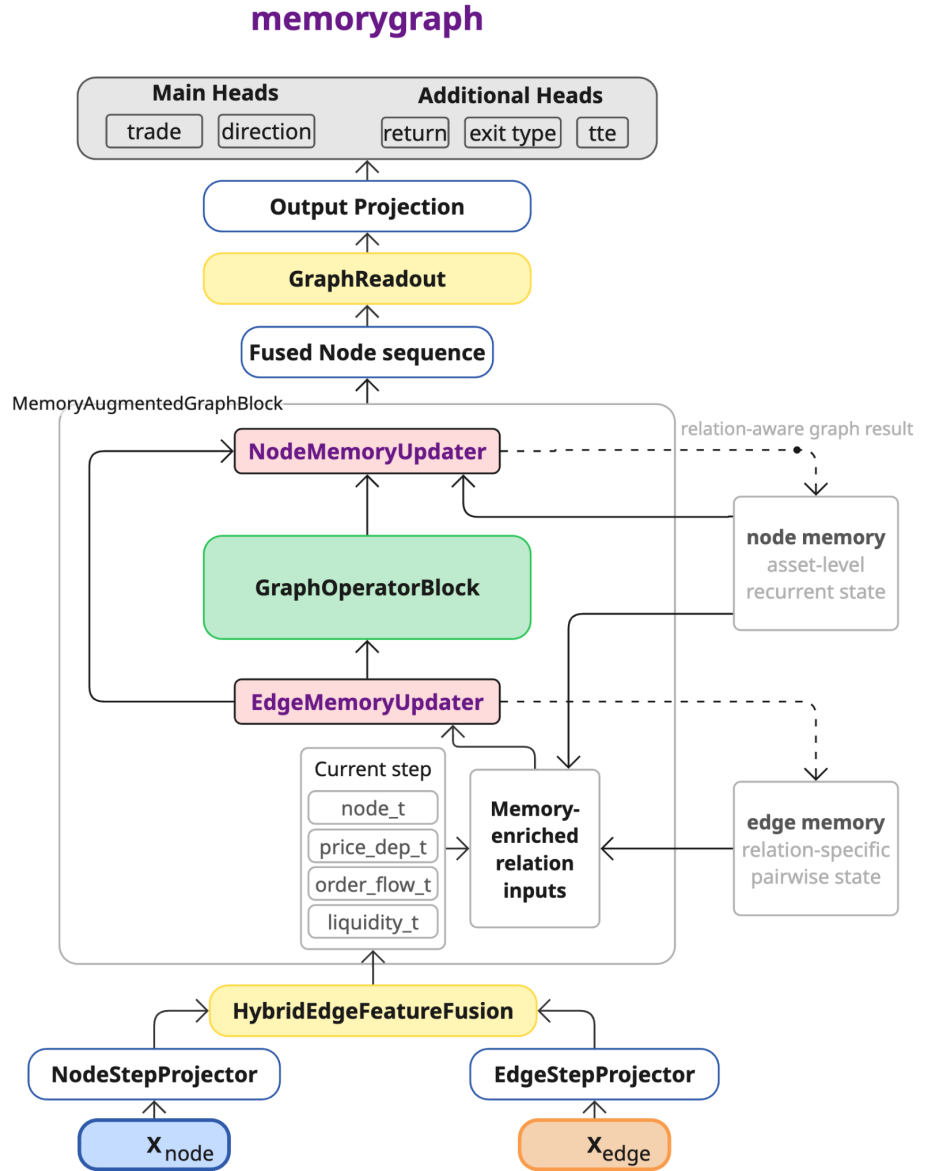


Figure 4.4. Detailed memorygraph architecture with recurrent node and edge memory.

Therefore, temporal modelling in this case is represented through *recurrent node and edge memory*, and graph interaction occurs *inside* the recurrent update loop.

4.5. Summary of Model Families

Taken together, the three model families define the architectural axes used in the empirical comparison: early relation fusion, late relation fusion, and recurrent relation-aware memory. This provides the structure for the Results chapter, where family design is evaluated jointly with operator choice, temporal frequency, and deployment-oriented model state.

5. Results

This chapter reports the empirical benchmark. The main evidence is the deployment-oriented **last-fold-M** comparison across all eighteen primary model-frequency configurations. The chapter then discusses frequency-specific outcomes, answers the research questions, compares selected **last-fold-M** and **final-refit-M** cases, and evaluates the hypotheses.

The main interpretive rule is that **net_pnl** is the primary economic outcome, **gross_pnl** indicates pre-cost signal extraction, and the **n_trades** shows how much turnover is required to obtain the result. The two AUC metrics (**trade_auc**, **dir_auc**) are also reported because the direction and trade heads are the decision-critical heads in the architecture, jointly determining whether the model trades and whether the position is long or short. An extended summary tables with **trade_rate**, **pnl_per_trade**, **sign_accuracy**, **win_rate**, **sharpe_like**, and **RMSE** are treated as supplementary operational diagnostics in Appendix C, while the raw results are also available at [github](#)². Those additional metrics helped a lot during modeling experiments to interpret hyperparameter tuning results and to understand whether the right direction is chosen for training.

The main platform for running experiments is Google Cloud Platform (GCP) - 91% of spent machine learning time, while initial attempts with smaller models were conducted on local machines (9% of machine time). Table 5.1 provides an actual machine learning time in hours, spent on GCP experiments (most experiments were executed on e2-standard-4 and e2-standard-16 virtual machines). Certain parts of the code were written with usage of Copilot and Codex tools, but the original idea is my own.

Table 5.1. Time spent on GNN experiments on Google Cloud Platform in hours.

Model family	1sec	1min	5min
Basegraph	108	405	10
Basegraph	34	406	10
Multigraph	42	234	5
Multigraph	124	237	5
Memorygraph	265	250	37
Memorygraph	236	265	36

Most of the time was spent on 1min experiments (1797 hours in total) because it is a tradeoff of data-depth complexity and execution time. While the model family related numbers show that **Memorygraph** experiments (1088 hours) were the most time-consuming, especially compared to **Multigraph** (647 hours), while **Basegraph** experiments were almost on the same level (974 hours).

5.1. Benchmark Overview

Table 5.1 reports the main **last-fold-M** benchmark. Within each frequency, the six models are directly comparable because they use the same input representation, target construction, validation logic, and event-based backtest. The 5min and 1min regimes are also directly comparable to each other because they solve the same 30-minute lookback / 5-minute horizon

² GitHub repository with raw results and code: github.com/vitalii-novikov/GNN_for_LOB

task. The 1sec regime should be interpreted as a high-frequency stress test with its own adapted task (2-minute lookback / 2-minute horizon). Appendix A summarizes the exact model-frequency grid, and Appendix B supports the comparison by documenting that the final blind intervals are chronologically aligned.

Table 5.1. *last-fold-M* benchmark overview across all model-frequency configurations.

Frequency	Model	gross pnl	net pnl	N trades	dir auc	trade auc
5min	base-gnn-conv	0.028156	0.020356	26	0.617105	0.700447
5min	base-gnn-mpnn	0.014415	0.006915	25	0.614912	0.727631
5min	multi-gnn-conv	0.012758	0.001958	36	0.616667	0.672097
5min	multi-gnn-mpnn	0.002941	-0.009359	41	0.625439	0.707304
5min	memory-gnn-conv	0.009459	0.004359	17	0.611842	0.734196
5min	memory-gnn-mpnn	-0.012463	-0.037363	83	0.537719	0.726026
1min	base-gnn-conv	0.059694	0.020094	132	0.525927	0.648157
1min	base-gnn-mpnn	-0.024239	-0.078539	181	0.504512	0.642097
1min	multi-gnn-conv	0.025767	-0.007833	112	0.541996	0.634787
1min	multi-gnn-mpnn	-0.003371	-0.035771	108	0.528405	0.645004
1min	memory-gnn-conv	-0.031247	-0.078947	159	0.479399	0.638928
1min	memory-gnn-mpnn	0.033605	0.009305	81	0.529844	0.635665
1sec	base-gnn-conv	0.139515	-0.094185	779	0.599753	0.841804
1sec	base-gnn-mpnn	0.052679	-0.065821	395	0.599093	0.848558
1sec	multi-gnn-conv	0.081710	-0.108790	635	0.597838	0.839544
1sec	multi-gnn-mpnn	0.079777	-0.080723	535	0.600538	0.868370
1sec	memory-gnn-conv	0.412032	-1.163268	5251	0.588785	0.490050
1sec	memory-gnn-mpnn	0.223788	-0.280512	1681	0.596713	0.863699

The table shows that positive net performance is concentrated in the slower shared-task regimes. The strongest model is `base-gnn-conv` at 5min, closely followed by `base-gnn-conv` at 1min, while every 1sec configuration is negative after costs. The 1sec rows are still informative because several models produce large positive gross PnL, but their trade counts are high enough to make the post-cost result negative. This result can also be seen on Figure 5.1, which provides a visual summary of the primary economic metric of the benchmark.

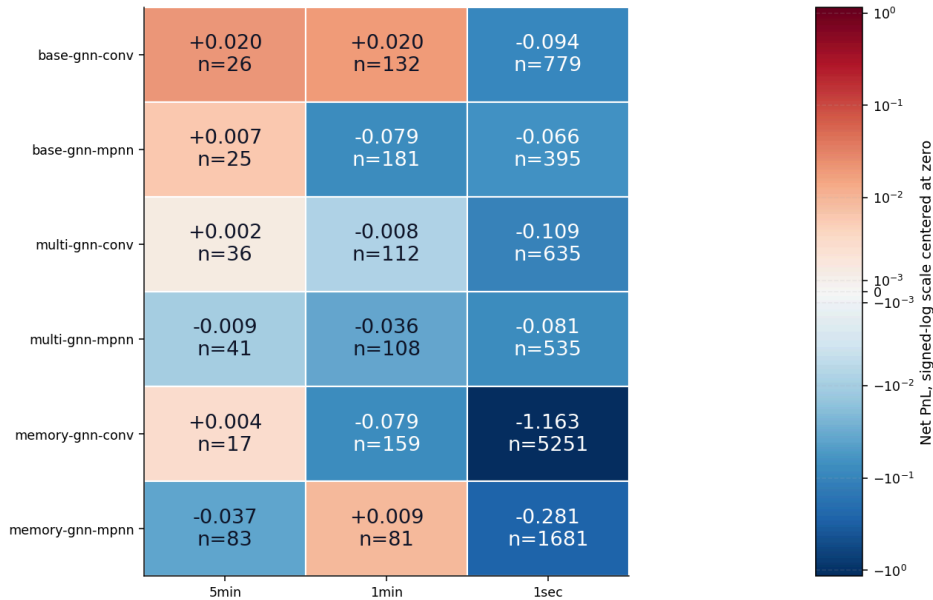


Figure 5.1. Benchmark overview by frequency, graph family, and operator.

The figure visualizes *last-fold-M* net PnL (`net_pnl`) across all eighteen model-frequency configurations, grouped by temporal resolution, graph family, and graph operator. The main

visual conclusion is that positive post-cost results are concentrated in the slower shared-task regimes, especially the `base-gnn-conv` specification, while every 1sec configuration remains negative after costs.

The next figure (Figure 5.2) moves the benchmark interpretation from endpoint totals to path-level behaviour. It shows the cumulative gross PnL paths for three representative final-holdout models: the best 5min model, the best 1min model, and the least-negative 1sec model after costs. The ETH midpoint index is included only as a market-context reference. The paths are an event-based visualization of when realized trade exits are added.

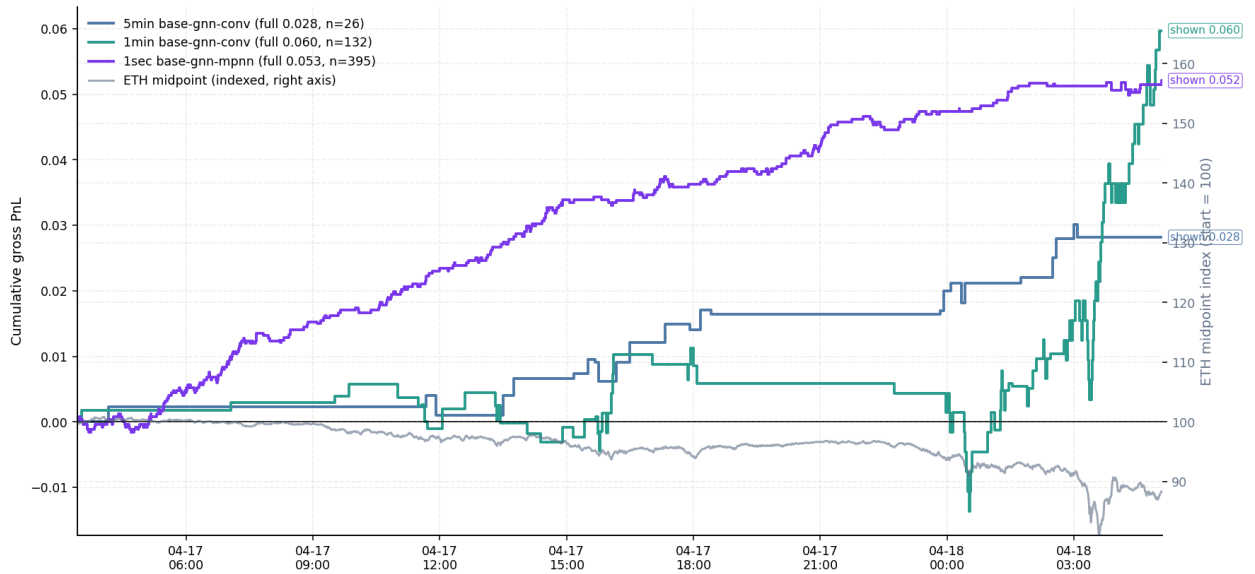


Figure 5.2. Cumulative gross PnL paths for representative final-holdout models.

Figure 5.2 indicates that the representative models do extract a pre-cost signal rather than merely tracking the displayed ETH market path. All three cumulative gross-PnL paths finish positive over a window in which the ETH midpoint index declines, which indicates that the gross PnL is not simply a passive exposure to a rising ETH market. However, the paths also show that this signal is obtained through different levels of trading intensity. The 5min `base-gnn-conv` path is sparse because it executes only 26 trades in the full final holdout. The 1min `base-gnn-conv` looks also sparse over the trading period, excluding the last part (starting about 04-18 00:00), when the overall activity of the ETH market also becomes more intense. As for the 1sec `base-gnn-conv` path, it shows a gradual change in positive direction, even when the ETH market looks stable. Overall the models show great results in terms of extracting useful information for profitable trading (before transaction costs), but this gross-PnL view establishes only the signal presence, but it does not yet establish deployability.

Figure 5.3 repeats the same comparison after transaction costs. This is the deployment-relevant version of the path comparison because the thesis treats `net_pnl` as the primary economic metric. The visual comparison clarifies why the gross result alone is insufficient. The 1min `base-gnn-conv` model extracts more pre-cost signal than the 5min `base-gnn-conv` model, but it also executes many more trades. As a result, the two models finish with almost identical net PnL despite very different gross PnL and turnover profiles.

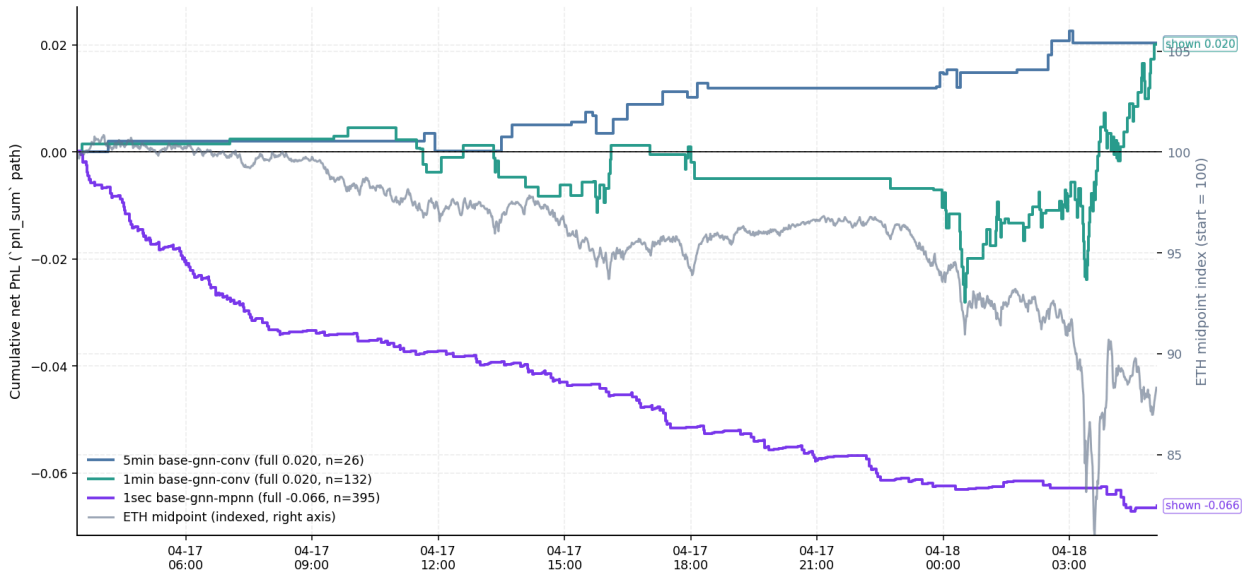


Figure 5.3. Cumulative net PnL paths for representative final-holdout models.

The contrast between Figure 5.2 and Figure 5.3 is central to the main interpretation. The 5min `base-gnn-conv` model ends at `net_pnl` = 0.020356 with 26 trades, while the 1min `base-gnn-conv` model ends at `net_pnl` = 0.020094 with 132 trades. The 1min model therefore finds more gross opportunity, but a larger part of that opportunity is consumed by the fixed round-trip cost proxy. The 5min model is less active but more efficient per trade. The 1sec `base-gnn-mpnn` model illustrates the same issue more strongly: its gross PnL is positive, but its net PnL ends at -0.065821 after costs.

Three patterns define the main results.

First, `base-gnn-conv` is the strongest deployment-oriented model at both shared-task frequencies. At 5min it achieves `net_pnl` = 0.020356, and at 1min it achieves `net_pnl` = 0.020094. Figure 5.3 strengthens this conclusion by showing that both positive results are path-level outcomes.

Second, the 1min regime should not be described as clearly superior in net economic terms. It is superior in gross signal extraction relative to the 5min representative model, but not in post-cost profitability. The correct interpretation is that the 1min regime reveals more trading signal, while the 5min regime converts a smaller amount of signal into a similarly strong net result through lower turnover.

Third, the 1sec regime separates great signal extraction from deployability. The strongest example is `memory-gnn-conv`, which reaches the largest gross result in the full benchmark (`gross_pnl` = 0.412032) but also executes 5251 trades and ends strongly negative after costs. Figure 5.2 shows that a positive gross signal exists at 1sec, but Figure 5.3 shows that this signal does not survive when the transaction costs are applied. The 1sec result is therefore not simply a failure of prediction. It is a failure of cost-aware selectivity under the current entry-policy benchmark.

5.2. Frequency-Specific Results

5.2.1. Five-Minute Regime

The 5min regime produces the clearest economically positive block of results. The best model is `base-gnn-conv`, with `gross_pnl` = 0.028156, `net_pnl` = 0.020356, and 26 trades. The second-best model is `base-gnn-mpnn`, with `net_pnl` = 0.006915 over 25 trades. The two baseline variants therefore occupy the top two economic positions.

The more complex families remain informative but not dominant. `multi-gnn-conv` is mildly positive, with `net_pnl` = 0.001958, while `multi-gnn-mpnn` is negative. `memory-gnn-conv` is also mildly positive, with `net_pnl` = 0.004359, while `memory-gnn-mpnn` is the weakest 5min model with `net_pnl` = -0.037363 and 83 trades.

The ranking metrics show why economic interpretation cannot rely on AUC alone. `multi-gnn-mpnn` has the highest `dir_auc` in the 5min block (0.625439), and `memory-gnn-conv` has the highest `trade_auc` (0.734196). Neither is the best economic model. The best deployable outcome comes from the baseline Conv model, which combines a positive gross signal with a modest number of trades and limited cost drag.

5.2.2. One-Minute Regime

The 1min regime is the richer shared-task stress test because it uses the same 30-minute lookback and 5min horizon as the 5min regime, but with more granular input information. The winner remains `base-gnn-conv`, with `gross_pnl` = 0.059694, `net_pnl` = 0.020094, and 132 trades.

This result is important because the gross signal is much larger than at 5min frequency (+112%), but the net result is almost identical (-1%). The reason is turnover: the 1min model extracts more pre-cost signal, but the larger number of trades absorbs most of the incremental edge through transaction costs. The 1min benchmark is therefore not stronger in strict net-profit terms, but it is stronger as a stress test of whether signal survives more active trading.

The second-best 1min model is `memory-gnn-mpnn`, with `net_pnl` = 0.009305 over 81 trades. This is the strongest shared-task result for memorygraph and suggests that recurrent memory can be useful at minute-level resolution. However, the result remains below the baseline winner.

The `multigraph` family does not produce positive net PnL at 1min frequency. `multi-gnn-conv` has positive gross PnL (0.025767) but ends at `net_pnl` = -0.007833; `multi-gnn-mpnn` is also negative. This does not show that relation-specific modelling contains no information. It shows that, under the present thresholding and cost assumptions, relation-specific information does not translate into superior net profitability.

5.2.3. One-Second Regime

The 1sec regime creates the sharpest separation between gross signal and net deployability. All 1sec models finish negative on `net_pnl`. The least-negative model is `base-gnn-mpnn` with `net_pnl` = -0.065821, followed by `multi-gnn-mpnn` with `net_pnl` = -0.080723, `base-gnn-conv` with `net_pnl` = -0.094185, and `multi-gnn-conv` with `net_pnl` = -0.108790. The `memorygraph` variants are substantially more negative after costs.

The gross results tell a different story. Figure 5.4 highlights this divergence between gross and net PnL in the 1sec regime.

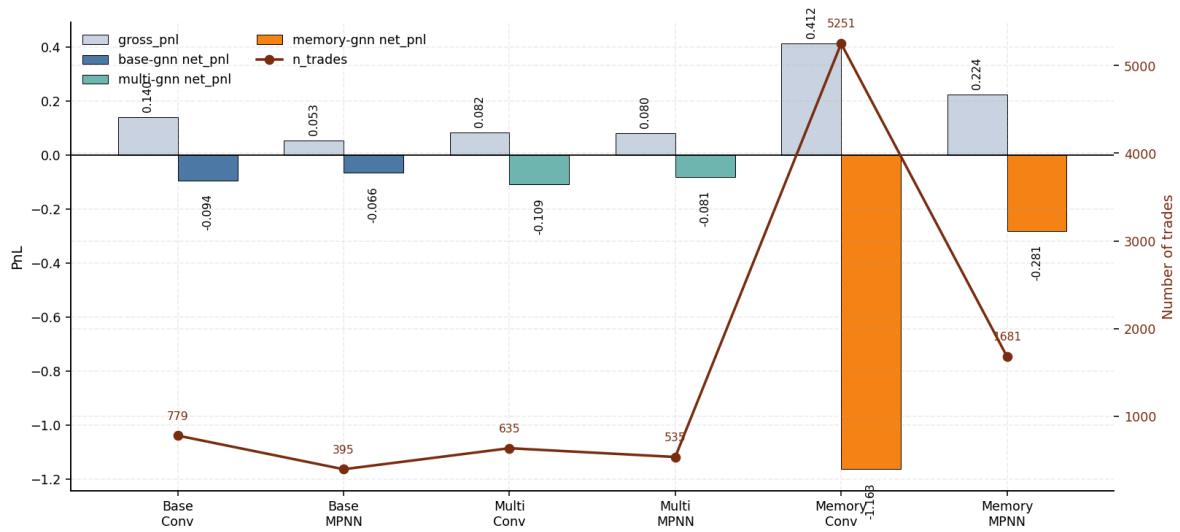


Figure 5.4 - Gross vs net PnL for 1sec models

Figure 5.4 shows that all 1sec models contain some pre-cost trading signal, but none of them preserve it after the transaction-cost proxy. The gap between the gross and net bars is therefore the key result of the figure: 1sec performance is dominated by cost drag rather than by the absence of directional information. `memory-gnn-conv` produces the largest gross signal in the entire benchmark, with `gross_pnl` = 0.412032, but it is also the clearest example of this cost problem. `memory-gnn-mpnn` produces the second-largest 1sec gross signal, with `gross_pnl` = 0.223788. These values show that the memory architecture is not failing to detect high-frequency structure: the problem is that its signal is too trade-intensive under the current cost model. This point is visible in Figure 5.5.

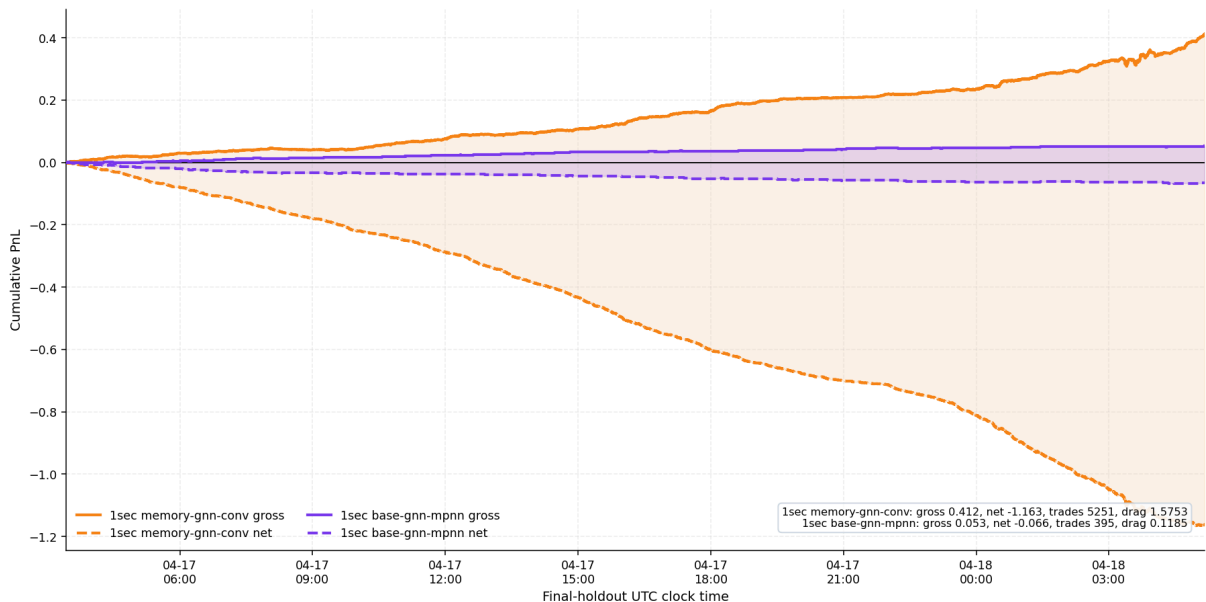


Figure 5.5. One-second cumulative gross-versus-net PnL paths for `memory-gnn-conv` and `base-gnn-mpnn`.

The solid orange line shows that `memory-gnn-conv` accumulates positive gross PnL over the final holdout. However, the dashed orange line declines strongly because the model executes 5251 trades. With the benchmark round-trip cost proxy of 0.0003, the implied cumulative cost

drag is approximately 1.5753, which is far larger than the model's gross PnL of 0.412032. The model therefore loses money after costs even though its pre-cost directional signal is positive.

The `base-gnn-mpnn` lines in Figure 5.5 provide a lower-turnover reference. This model also has positive gross PnL and negative net PnL, but its cost drag is much smaller because it executes 395 trades rather than 5251. Its implied cost drag is approximately 0.1185, compared with 1.5753 for `memory-gnn-conv`. Therefore, both models face the same cost problem, while the high-turnover memory model is punished much more significantly.

The Appendix C diagnostics support the same interpretation. The 1sec `memory-gnn-conv` result has `trade_rate` = 0.056665, which is far above the `trade_rate` = 0.004263 of the 1sec `base-gnn-mpnn` result. Its `sign_accuracy` = 0.584841 is not poor in isolation, but its `pnl_per_trade` = -0.000222 and `sharpe_like` = -28.865252 show that small post-cost losses accumulate rapidly under high turnover.

The 1sec evidence is central to the thesis. It supports a two-layer conclusion: `memorygraph` models can extract high-frequency gross signals (which is the strongest compared to 5min and 1min regimes), but this signal is not sufficiently selective under the current benchmark. Therefore, the main unresolved problem for the 1sec regime is conversion of high-frequency signals into sparse, stable, and cost-aware trading decisions.

5.2.4. Summary of Frequency-Specific Results

Across the three frequency regimes, the 5min setting is the most efficient post-cost shared-task benchmark, while the 1min setting reveals richer gross signal without a superior net outcome. The 1sec setting operates as a turnover and cost-drag stress test, which motivates the research-question answers that follow.

5.3. Answer to RQ1: Which Graph Family Performs Best?

The answer to RQ1 is that `basegraph` performs best overall under the controlled entry-model benchmark.

The strongest evidence comes from the two shared-task regimes. At 5min, the best model is `base-gnn-conv` with `net_pnl` = 0.020356. At 1min, the best model is again `base-gnn-conv`, with `net_pnl` = 0.020094. Both results are positive, and both are obtained by the same family and operator.

At 1sec, no family produces positive net PnL. This means the high-frequency regime cannot be used to identify a robust deployment winner. Instead, it shows that all families face a cost and turnover barrier under the current entry policy.

The family-level conclusion is therefore conservative but clear. Under fixed targets, fixed features, fixed validation, fixed thresholds, and fixed event-based exits, the simpler single-graph baseline is the most reliable architecture. The richer families may contain useful signal, but they do not produce a stronger post-cost benchmark result.

5.4. Answer to RQ2: How Important Is the Conv-versus-MPNN Operator Choice?

The operator choice is important, but its effect is not universal.

At 5min, Conv outperforms MPNN on **net_pnl** in all three families:

1. basegraph: 0.020356 versus 0.006915.
2. multigraph: 0.001958 versus -0.009359.
3. memorygraph: 0.004359 versus -0.037363.

At 1min, Conv remains better for **basegraph** and **multigraph**, but **memorygraph** reverses in favor of MPNN:

1. basegraph: 0.020094 versus -0.078539.
2. multigraph: -0.007833 versus -0.035771.
3. memorygraph: -0.078947 versus 0.009305.

At 1sec, all models are net negative, but MPNN is less negative than Conv in each family:

1. basegraph: -0.065821 versus -0.094185.
2. multigraph: -0.080723 versus -0.108790.
3. memorygraph: -0.280512 versus -1.163268.

The operator therefore changes economic outcomes significantly. The strongest shared-task model is Conv-based, but the least negative 1sec models are MPNN-based. The conclusion is that Conv is not always better, nor MPNN is always better. The operator must be selected jointly with the model family, frequency regime, and cost-sensitive trading policy.

5.5. Answer to RQ3: How Does Temporal Resolution Affect Relation and Memory Mechanisms?

The results do not support the hypothesis that finer temporal resolution automatically increases the net economic value of richer graph mechanisms.

For **multigraph**, relation-specific processing does not beat the baseline on **net_pnl** at any frequency. At 5min frequency, it is mildly positive in the Conv variant but below the baseline. At 1min frequency, both variants are net negative. At 1sec frequency, both variants have positive gross signal but negative net PnL.

For **memorygraph**, the answer is more nuanced. The family becomes most distinctive at 1sec frequency, exactly where stateful memory was expected to matter most. Its gross results are the largest in the benchmark. This provides partial evidence that recurrent memory can surface high-frequency opportunities. However, the same results show that memory also produces excessive trading activity under the current policy. The net effect is strongly negative.

The best interpretation is therefore two-layered. Finer temporal resolution appears to increase the amount of extractable short-horizon signal, especially for memory-based models. At the same time, it increases the penalty for insufficient trade selectivity. In the current benchmark, the cost and turnover effect dominates the signal-extraction effect.

5.6. Answer to RQ4: Are Conclusions Stable Between last-fold-M and final-refit-M?

The deployment-state comparison shows that **last-fold-M** and **final-refit-M** are related but not interchangeable. The **last-fold-M** state remains the main deployment reference because it is produced by the final chronological cross-validation fold. The **final-refit-M** state is useful because it tests what happens when a model is refitted on a larger pre-holdout sample, but it does not replace the chronological evidence. The conceptual distinction between these states is summarized in Figure 5.6.

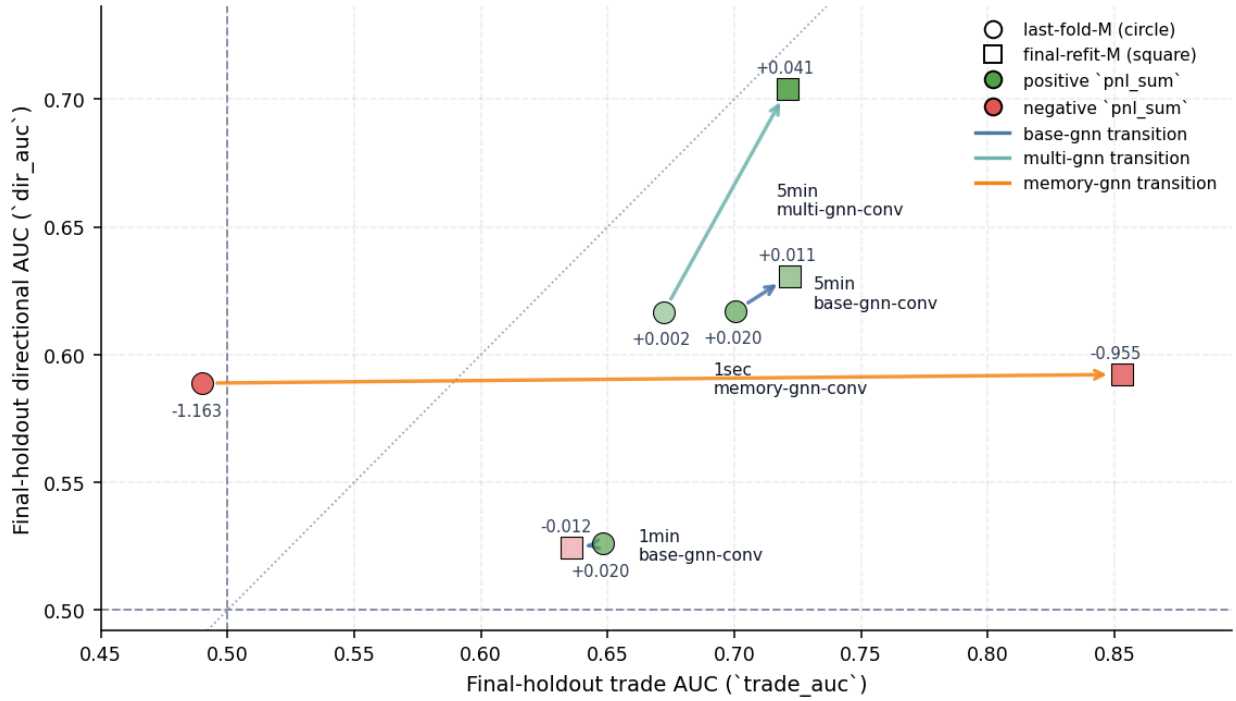


Figure 5.6 *'last-fold-M' versus 'final-refit-M' as deployment-oriented model states*

The figure shows that refitting is informative but not mechanically beneficial. Several **final-refit-M** points improve ranking diagnostics, yet the economic arrows do not move uniformly upward: the 5min **multigraph** case improves strongly, the 1min baseline worsens, and the selected 1sec memory case remains negative after costs. The selected comparisons in Sections 5.6.1-5.6.4 therefore treat **final-refit-M** as a larger-sample robustness comparison, while **last-fold-M** remains the primary deployment-oriented state defined in Section 3.9.

The deeper comparison of the model states for selected models is provided in sub-sections 5.6.1-5.6.4, while the information about all models with all states is provided in Appendix C.

5.6.1. Best Five-Minute Model: base-gnn-conv

Table 5.2 reports the **last-fold-M** and **final-refit-M** comparison for the best 5min model (**base-gnn-conv**).

Table 5.2. The best five-minute model states deployment-state comparison

Frequency	Training cycle	gross_pnl	net_pnl	n_trades	dir_auc	trade_auc
5min	last-fold-M	0.028156	0.020356	26	0.617105	0.700447
5min	final-refit-M	0.01757	0.01127	21	0.630702	0.721795
		160%	181%		98%	97%

The line with percent values shows how good was **last-fold-M** compared to **final-refit-M** because we assume that the later model state is the 100% potential of the current model. In that particular case we see that the refit on larger samples improves AUCs, but the main economic metrics are better for the smaller (**last-fold-M**) model. But at least, **base-gnn-conv** remains positive after refitting. Its net PnL declines, but its ranking metrics improve. This is the cleanest deployment-stability case in the selected comparisons. It also shows why final refitting should not be assumed to improve the main economic metric: more training data improve AUC here, but not **net_pnl**.

5.6.2. Best One-Minute Model: base-gnn-conv

Table 5.3 reports the **last-fold-M** and **final-refit-M** comparison for the best 1min model (**base-gnn-conv**).

Table 5.3. The best one-minute model deployment-state comparison.

Frequency	Training cycle	gross_pnl	net_pnl	n_trades	dir_auc	trade_auc
1min	last-fold-M	0.059694	0.020094	132	0.525927	0.648157
1min	final-refit-M	0.007198	-0.012002	64	0.524286	0.635712
		829%	+		100%	102%

The 1min winner is less stable. The **last-fold-M** model is clearly positive, while the **final-refit-M** version turns negative. The AUC values change only modestly, which suggests that the underlying ranking quality remains similar while the score-to-trade conversion becomes less economically favorable. This case reinforces the deployment argument: a model can look similar in predictive diagnostics but materially different in realized trading performance.

5.6.3. Selected One-Second Memorygraph Case: memory-gnn-conv

Table 5.4 reports the **last-fold-M** and **final-refit-M** comparison for the **memory-gnn-conv**, which achieved the largest **gross_pnl** amount among all models.

Table 5.4. The most gross-profitable one-second model deployment-state comparison.

Frequency	Training cycle	gross_pnl	net_pnl	n_trades	dir_auc	trade_auc
1sec	last-fold-M	0.412032	-1.163268	5251	0.588785	0.490050
1sec	final-refit-M	0.443031	-0.954969	4660	0.592186	0.852874
		93%	-		99%	57%

The selected 1sec case is the most informative high-frequency stress example. Refitting increases **gross_pnl**, reduces the trade count, significantly improves **trade_auc**, and makes the **net_pnl** less negative. Nevertheless, the model remains strongly unprofitable after costs. The

central 1sec conclusion therefore survives refitting: memory-based high-frequency signal is present, but it is not sufficiently selective under the current benchmark.

5.6.4. Informative Five-Minute Refit Case: multi-gnn-conv

Table 5.5 reports the `last-fold-M` and `final-refit-M` comparison for the `multi-gnn-conv`, which achieved the largest `dir_auc` among all models.

Table 5.5. The most direction-oriented five-minute model deployment-state comparison.

Frequency	Training cycle	gross_pnl	net_pnl	n_trades	dir_auc	trade_auc
5min	<code>last-fold-M</code>	0.012758	0.001958	36	0.616667	0.672097
5min	<code>final-refit-M</code>	0.061167	0.041367	66	0.703947	0.72092
		21%	5%		88%	93%

This case is analytically important because the `final-refit-M` version of `multi-gnn-conv` reaches `dir_auc` > 70% and a strongly positive `net_pnl`. It shows that relation-preserving graph processing can become highly effective under a larger-sample refit. The percentages in the table show that the smaller version achieves significantly weaker results especially for economic metrics. In the deployment-oriented state, `multi-gnn-conv` is only weakly positive and remains below `base-gnn-conv`. The case should therefore be interpreted as evidence of refit sensitivity and potential future value, not as proof that `multigraph` is already the strongest deployable family.

5.6.5. Summary of Answer to RQ4

Overall, the selected state comparisons show that refitting can improve diagnostics and can sometimes improve `net_pnl`, but it does not uniformly improve deployability. The mixed economic response means that `last-fold-M` remains the deployment-oriented benchmark reference. The `final-refit-M` state instead provides complementary evidence about larger-sample sensitivity before the hypothesis assessment summarizes this distinction.

5.7. Hypothesis Assessment

H1. The one-minute regime should be the strongest shared-task benchmark.

This hypothesis is not supported on the primary economic metric. The strongest 1min model reaches `net_pnl` = 0.020094, while the strongest 5min model reaches `net_pnl` = 0.020356. The difference is small, but the hypothesis predicts 1min superiority, which is not observed. Moreover, the PnL paths analysis reveals that the 5min regime outperforms most of the time on the observed holdout interval. The 1min regime remains important because it produces more trades and a much larger gross signal, but it is not the strongest shared-task regime in strict net terms.

H2. Explicit multi-relation modelling should outperform the simpler baseline more clearly at finer resolutions.

This hypothesis is not supported on the primary economic benchmark. `multigraph` does not beat `basegraph` on `net_pnl` at 5min, 1min, or 1sec frequency. The interesting `multi-gnn-conv` final-refit case provides evidence that relation-specific modelling can become strong under

some training states, but the deployment-oriented **last-fold-M** benchmark does not support a general **multigraph** advantage.

H3. Stateful memory should become more valuable as the market is observed more finely.

This hypothesis is partially supported for gross signal extraction but not supported for net deployable performance. At 1sec frequency, **memorygraph** produces the strongest gross PnL values in the study. However, both memory variants remain net negative, and **memory-gnn-conv** is especially negative because of excessive turnover. Memory helps reveal short-lived signals, but the current benchmark does not show that it improves post-cost profitability.

H4. Conv and MPNN operators should not be uniformly dominant across families.

This hypothesis is supported. Conv dominates the 5min net results and produces the strongest shared-task model overall. At 1min frequency, Conv remains better for **basegraph** and **multigraph**, while MPNN is better for **memorygraph**. At 1sec frequency, MPNN is less negative in all families on net PnL. The operator effect is therefore economically meaningful and frequency-dependent.

*H5. **last-fold-M** and **final-refit-M** should provide broadly consistent but not necessarily equivalent evidence.*

This hypothesis is partially supported. The selected comparisons do not overturn the broad conclusion that **basegraph** is the most reliable deployment-oriented family and that 1sec models remain net negative. However, the model-level story can change materially, and the economic response is mixed rather than mechanically positive. The 1min **base-gnn-conv** turns negative after refitting, while the 5min **multi-gnn-conv** becomes very strong after refitting and several diagnostics improve. The evidence therefore supports reporting both states transparently and prioritizing **last-fold-M** for deployment interpretation.

5.8. Summary of Research Question Answers

Table 5.6 consolidates the answers to the four research questions. The table is included to make the empirical storyline explicit before the thesis moves from results to discussion.

Table 5.6. Summary of answers to the research questions.

Research question	Main empirical answer	Key evidence	Interpretation
RQ1. Which graph family performs best under a controlled entry-model benchmark?	basegraph performs best overall.	base-gnn-conv is the strongest last-fold-M model at both 5min and 1min, with net_pnl = 0.020356 and net_pnl = 0.020094, respectively.	The simpler single-graph baseline is the most reliable architecture under the tested benchmark.
RQ2. How important is the Conv-versus-MPNN operator choice?	Operator choice materially changes outcomes, but no operator is universally dominant.	Conv is strongest in the shared-task winners, while MPNN is less negative for all families at 1sec.	Operator choice should be evaluated jointly with model family, frequency, and cost-sensitive thresholding.
RQ3. How does temporal resolution affect relation and memory mechanisms?	Higher temporal resolution increases gross signal visibility but also increases turnover and cost pressure.	memory-gnn-conv has the largest 1sec gross signal, but it is strongly net negative because it executes 5251 trades.	Finer data can reveal short-lived signal, but selectivity becomes the main bottleneck.
RQ4. Are conclusions stable between last-fold-M and final-refit-M ?	The broad story is stable, but model-level economics can change substantially.	5min base-gnn-conv remains positive after refit, 1min base-gnn-conv turns negative, and 5min multi-gnn-conv becomes very strong after refit.	final-refit-M is informative as a larger-sample comparison, but last-fold-M remains the primary deployment-oriented reference.

5.9. Summary of Hypothesis Assessment

Table 5.7 summarizes the hypothesis assessment. This summary separates unsupported, partially supported, and supported claims and clarifies which conclusions are based on net economic outcomes rather than only on predictive diagnostics.

Table 5.7. Summary of hypothesis assessment.

Hypothesis	Status	Main reason
H1. The 1min regime should be the strongest shared-task benchmark.	Not supported	The best 1min net result (net_pnl = 0.020094) is slightly below the best 5min net result (net_pnl = 0.020356).
H2. Explicit multi-relation modelling should outperform the simpler baseline more clearly at finer resolutions.	Not supported	multigraph does not beat basegraph on last-fold-M net_pnl at 5min, 1min, or 1sec.
H3. Stateful memory should become more valuable as the market is observed more finely.	Partially supported	memorygraph produces the strongest 1sec gross signal, but it remains strongly negative after costs.
H4. Conv and MPNN operators should not be uniformly dominant across families.	Supported	Conv dominates the shared-task winners, while MPNN is less negative across all families at 1sec.
H5. last-fold-M and final-refit-M should provide broadly consistent but not necessarily equivalent evidence.	Partially supported	The broad deployment conclusion remains cautious, while diagnostics and individual model outcomes can change materially after refitting without implying uniform ' net_pnl ' improvement.

6. Discussion

6.1. Main Findings by Research Question

The main finding is that architectural complexity did not guarantee better trading outcomes. Under the strict shared-task benchmark, the simpler `basegraph` family is strongest, and the winning specification is `base-gnn-conv`. This result is methodologically important because it was obtained under a common target construction, common output interface, common thresholding framework, and common event-based backtest.

For RQ1, the strongest family is `basegraph`. For RQ2, the operator effect is reliable and conditional, but Conv is the stronger shared-task choice. For RQ3, finer temporal resolution increases the visibility of gross high-frequency signal, especially for `memorygraph`, but it also increases cost drag and turnover risk. For RQ4, `last-fold-M` and `final-refit-M` provide complementary but non-equivalent evidence: the latter can improve diagnostics, but it does not uniformly improve `net_pnl`.

A central theme is that predictive quality, gross signal extraction, and net profitability must be separated. The 1sec `memorygraph` results make this most visible. A model can generate a large positive gross PnL and still fail economically because the used transaction costs overtake the most trades, even while the gross signal is noticeable. Conversely, a model with less dramatic ranking statistics can be more useful if it is more selective (focuses on larger profit executions).

6.2. Comparison with Previous Work

The results are consistent with the broader LOB literature in one respect: short-horizon market data contain learnable structure [5], [6], [7], [11]. The presence of positive gross PnL in several models, especially at 1sec, supports the idea that order book and cross-asset microstructure features contain information about subsequent movement.

However, the results also qualify optimistic interpretations of deep learning for market prediction. DeepLOB and related neural LOB studies emphasize the ability of convolutional, recurrent, and attention-based models to learn from high-dimensional order book states [7], [9], [11]. Recent work such as HLOB further suggests that structured representations can capture information persistence inside the book [10].

The graph-learning literature motivates the use of relation-aware models, and recent financial graph studies motivate multi-relational dynamic modelling [13], [14], [15], [16], [17], [18], [21], [22], [19], [20]. The present findings are more cautious. Multi-relation modelling is plausible and sometimes useful, but in the primary `last-fold-M` benchmark it does not beat the simpler baseline. This does not contradict the graph literature; rather, it shows that graph complexity must be evaluated against the specific economic objective, market regime, asset universe, and cost model.

The temporal graph and memory literature also provides a useful comparison [17], [18], [19]. Temporal memory is designed to preserve information across changing graph states. The `memorygraph` results support this idea before costs: the largest 1sec gross signals come from memory-based models. Yet they also show that memory without sufficiently selective trading

control can amplify turnover. In market microstructure, the ability to detect many short-lived opportunities is not enough if the opportunities are too small relative to costs.

6.3. Scientific Implications

The scientific implication is that graph-based market microstructure research should evaluate architecture under deployment-aware metrics (in addition to predictive metrics). AUC, accuracy, and regression error remain useful, but they do not settle whether a model is economically meaningful. The benchmark must include trade count, gross PnL, net PnL, and cost sensitivity.

A second implication is that relation modelling should be treated as an empirical design choice rather than an assumed improvement. Preserving multiple relation channels is theoretically attractive, especially in finance, but the main benchmark shows that early fusion in a simpler baseline can outperform late relation-specific processing under some conditions.

A third implication concerns temporal resolution. Higher frequency can increase the quantity of detectable signal, but it also raises the cost of acting on that signal. Therefore evaluating high-frequency models with explicit turnover and cost analysis risks is crucial, which was also done in this work.

6.4. Practical and Deployment Implications

The strongest deployment-oriented evidence favors `base-gnn-conv` in the shared-task regimes. The model is not a production trading system; it is the best entry model in a controlled final-holdout benchmark. A real deployment would require latency modelling, exchange-specific fee and slippage assumptions, monitoring for regime drift, capital constraints, live order placement logic, risk controls, and operational monitoring.

`last-fold-M` is deployment-relevant because it resembles the chronological situation of using the latest available model on an unseen future segment. `final-refit-M` adds information about whether a larger training sample changes the holdout result. The 5min `multi-gnn-conv` example demonstrates that a model that is weak in `last-fold-M` can become strong under final refit. But still, the final refit state is used as an theoretically “expected” model, which allows us to compare on how well the deployment-oriented model was.

The main results imply that realistic deployment should prioritize selectivity and cost robustness. The high-frequency memory models are not suitable for deployment in their current form because their gross edge is overwhelmed by turnover. Future deployment-oriented work should therefore evaluate trade-rate controls, threshold stability, cost sensitivity, and no-trade calibration before treating high-frequency graph signals as practically useful.

From an impact perspective, the immediate value of the thesis is methodological rather than operational. It provides a reproducible benchmark for comparing graph architectures under chronological validation and transaction costs. Potential positive impacts include better model-selection discipline, clearer reporting of high-frequency ML limitations, and reduced risk of over-interpreting ranking metrics. Potential negative impacts include encouraging automated trading experiments with simplified risk controls if the results are misread as live-profitability evidence.

6.5. Limitations, Weaknesses, and Sources of Bias

Several limitations affect the interpretation of the thesis. They do not invalidate the controlled benchmark, but they define the scope within which the empirical conclusions should be read.

First, the empirical scope is narrow. The graph contains only ADA, BTC, and ETH, with ETH as the sole target asset. This improves interpretability and keeps the architecture comparison controlled, but it creates asset-selection bias. The results may not transfer to larger crypto universes, equities, futures, foreign exchange, or less liquid instruments. A larger universe could also change the value of **multigraph**, because more assets would create more relation pathways. It would also require a separate data-engineering layer for obtaining comparable LOB data from exchanges or paid third-party APIs, aligning clocks, handling missing levels and venue differences, and modelling venue-specific costs.

Second, the conclusions may depend on the sampled market regime. The final holdout is chronological and therefore more realistic than a random split, but it still represents one late segment of the available data. If this period has unusual volatility, liquidity, or directional structure, the measured ranking of architectures may partly reflect that temporal slice. A broader study would repeat the benchmark across several calendar periods, volatility regimes, and market states.

Third, the target and threshold design can shape the measured performance. Triple-barrier labels depend on barrier widths, rolling volatility estimates, timeout handling, and direction-label masking. In addition, trade and direction thresholds are selected from finite validation grids and then applied to the final holdout. These choices are shared across models and protect comparability, but they can still influence which architectures appear more selective or more profitable, especially in the one-second regime.

Fourth, the transaction-cost and execution model is deliberately simplified. The benchmark uses a constant round-trip cost proxy and a sequential event-based backtest. This is sufficient for controlled model comparison, but it is not a live execution simulator. Real deployment would require exchange fees, spreads, slippage, queue priority, latency, market impact, adverse selection, risk limits, and operational monitoring. The one-second results are especially sensitive to this limitation because turnover dominates post-cost outcomes.

Finally, the architecture and tuning space is limited. The thesis compares three graph families and two graph-operator styles under a fair but non-exhaustive training setup. It does not test transformer-based LOB models, hybrid attention-GNN architectures, reinforcement-learning exits, probabilistic calibration layers, or explicitly turnover-regularized objectives. Broader hyperparameter search could also change the relative strength of the multigraph and memorygraph families.

7. Conclusions and Future Research

7.1. Overall Conclusion

This thesis evaluated how graph family, graph operator, temporal resolution, and deployment-oriented model state affect short-horizon limit order book prediction under a controlled, friction-aware benchmark. The main empirical answer is conservative. The strongest `last-fold-M` evidence favors the simpler basegraph family, specifically `base-gnn-conv`, at both shared-task frequencies.

At 5min frequency, `base-gnn-conv` achieves the best `last-fold-M` net result with `net_pnl` = 0.020356 over 26 trades. At 1min frequency, `base-gnn-conv` again achieves the best result with `net_pnl` = 0.020094 over 132 trades. At 1sec frequency, all models are net negative after transaction costs. The core high-frequency finding is therefore the divergence between gross signal and net deployability. `memory-gnn-conv` produces the largest 1sec gross signal, but it fails after costs because the trading policy expresses that signal through excessive turnover.

The thesis shows that under a fair entry-model benchmark, additional relation-specific processing and recurrent memory change the diagnostic and trading profile of the models, but they do not automatically translate into stronger post-cost trading performance. The strongest architecture is the one that best balances signal extraction, selectivity, and turnover.

The deployment-state analysis reinforces this conclusion. `last-fold-M` should remain the primary deployment reference because it respects chronological model selection. `final-refit-M` is useful as a possibly achievable reference in terms of economic outcomes. The 5min baseline refit remains positive, the 1min baseline refit turns negative, and the 5min `multi-gnn-conv` refit becomes very strong.

The final thesis conclusion is: richer graph architectures (`multigraph` and `memorygraph`) extract useful signals in some cases, but they do not establish a robust post-cost advantage over the simpler `basegraph` benchmark under the tested dataset, model families, and evaluation protocol. The main methodological implication is that architecture quality cannot be judged separately from the trading-oriented evaluation layer. In this benchmark, the decisive requirement is the ability to translate relational and temporal signals into selective, stable, and cost-aware entry decisions.

7.2. Future Research

Future research should focus first on turnover-aware modelling. The 1sec experiments show that memory-based graph models can identify many short-lived opportunities, but they lack sufficient selectivity. Future work should investigate cost-aware objectives, stricter no-trade calibration, sparse event-driven state updates, confidence-aware memory resets, and threshold policies that explicitly penalize excessive trading.

A second direction is execution-aware evaluation. The present benchmark fixes a common realized-event exit rule in order to preserve fairness. Future work could keep the common entry benchmark for comparability and then test the strongest entry models under adaptive exits, richer slippage assumptions, exchange-specific fees, latency constraints, queue-position modelling, partial-fill assumptions, and adverse-selection scenarios.

A third direction is larger-universe graph modelling. A three-asset graph is useful for a controlled thesis benchmark, but it may understate the value of relation-specific architectures. Adding more crypto assets, stablecoins, sector proxies, derivatives, or cross-venue liquidity measures would create a stronger test of whether **multigraph** becomes more valuable when the relation space is richer.

A fourth direction is selective memory. The current **memorygraph** models appear capable of finding high-frequency gross signal but not of controlling trade frequency. Future models should examine sparse memory updates, event-triggered memory writes, confidence-aware state resets, and memory mechanisms coupled to explicit trade-rate constraints.

A fifth direction is robustness and uncertainty analysis. Future versions of the benchmark should report fold-level dispersion, bootstrap confidence intervals for economic metrics, pairwise model-comparison tests, regime-specific performance summaries, drawdown statistics, and cost-sensitivity curves. These additions would make it easier to distinguish genuine architecture effects from temporal-slice effects or threshold-selection effects.

A final direction is systematic cost sensitivity. Because the main high-frequency weakness is the gap between gross and net performance, future research should report cost-sensitivity curves across fee, spread, slippage, and latency assumptions. This would clarify whether a model is close to viability under realistic execution improvements or whether its signal is too small relative to unavoidable trading frictions.

References

- [1] Cont, R., Stoikov, S., & Talreja, R. (2010). *A stochastic model for order book dynamics*. Operations Research, 58(3), 549-563. [Link](#)
- [2] Gould, M. D., Porter, M. A., Williams, S., McDonald, M., Fenn, D. J., & Howison, S. D. (2013). *Limit order books*. Quantitative Finance, 13(11), 1709-1742. [Link](#)
- [3] Cont, R. (2001). *Empirical properties of asset returns: stylized facts and statistical issues*. Quantitative Finance, 1(2), 223-236. [Link](#)
- [4] Schnaubelt, M., Rende, J., & Krauss, C. (2019). *Testing stylized facts of Bitcoin limit order books*. Journal of Risk and Financial Management, 12(1), 25. [Link](#)
- [5] Ntakaris, A., Magris, M., Kannianen, J., Gabbouj, M., & Iosifidis, A. (2018). *Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods*. Journal of Forecasting, 37(8), 852-866. [Link](#)
- [6] Sirignano, J., & Cont, R. (2019). *Universal features of price formation in financial markets: perspectives from deep learning*. Quantitative Finance, 19(9), 1449-1459. [Link](#)
- [7] Zhang, Z., Zohren, S., & Roberts, S. (2019). *DeepLOB: Deep convolutional neural networks for limit order books*. IEEE Transactions on Signal Processing, 67(11), 3001-3012. [Link](#)
- [8] Arroyo, Á., Cartea, Á., Moreno-Pino, F., & Zohren, S. (2024). *Deep attentive survival analysis in limit order books: Estimating fill probabilities with convolutional-transformers*. Quantitative Finance, 24(1), 35-57. [Link](#)
- [9] Jung, J., & Lee, K. (2025). *Attention-based reading, highlighting, and forecasting of the limit order book*. Quantitative Finance, 25(7), 1015-1027. [Link](#)
- [10] Briola, A., Bartolucci, S., & Aste, T. (2025). *HLOB: Information persistence and structure in limit order books*. Expert Systems with Applications, 266, 126078. [Link](#)
- [11] Sirignano, J. (2019). *Deep learning for limit order books*. Quantitative Finance, 19(4), 549-570. [Link](#)
- [12] López de Prado, M. (2018). *Advances in Financial Machine Learning*. Wiley. ISBN: 978-1-119-48208-6. [Link](#)
- [13] Kipf, T. N., & Welling, M. (2017). *Semi-supervised classification with graph convolutional networks*. International Conference on Learning Representations. [Link](#)
- [14] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). *Graph attention networks*. International Conference on Learning Representations. [Link](#)

- [15] Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Yu, P. S. (2020). *A comprehensive survey on graph neural networks*. IEEE Transactions on Neural Networks and Learning Systems, 32(1), 4-24. [Link](#)
- [16] Wu, Z., Pan, S., Long, G., Jiang, J., Chang, X., & Zhang, C. (2020). *Connecting the dots: Multivariate time series forecasting with graph neural networks*. Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 753-763. [Link](#)
- [17] Kazemi, S. M., Goel, R., Jain, K., Kobyzev, I., Sethi, A., Forsyth, P., & Poupart, P. (2020). *Representation learning for dynamic graphs: A survey*. Journal of Machine Learning Research, 21(70), 1-73. [Link](#)
- [18] Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., & Bronstein, M. (2020). *Temporal graph networks for deep learning on dynamic graphs*. arXiv preprint arXiv:2006.10637. [Link](#)
- [19] Zheng, Y., Yi, L., & Wei, Z. (2025). *A survey of dynamic graph neural networks*. Frontiers of Computer Science, 19, Article 196323. [Link](#)
- [20] Corradini, F., Gerosa, F., Gori, M., Lucheroni, C., Piangerelli, M., & Zannotti, M. (2026). *A systematic literature review of spatio-temporal graph neural network models for time series forecasting and classification*. Neural Networks, 195, 108269. [Link](#)
- [21] Wang, J., Zhang, S., Xiao, Y., & Song, R. (2022). *A review on graph neural network methods in financial applications*. Journal of Data Science, 20(2), 111-134. [Link](#)
- [22] Qian, H., Zhou, H., Zhao, Q., Chen, H., Yao, H., Wang, J., Liu, Z., Yu, F., Zhang, Z., & Zhou, J. (2024). *MDGNN: Multi-relational dynamic graph neural network for comprehensive and dynamic stock investment prediction*. Proceedings of the AAAI Conference on Artificial Intelligence, 38(13), 14642-14650. [Link](#)
- [23] Cartea, Á., Jaimungal, S., & Penalva, J. (2015). *Algorithmic and High-Frequency Trading*. Cambridge University Press. ISBN: 978-1-107-09114-6. [Link](#)
- [24] Martinsn. *High-Frequency Crypto Limit Order Book Data*. Kaggle dataset. Dataset page: <https://www.kaggle.com/datasets/martinsn/high-frequency-crypto-limit-order-book-data>

List of Figures

- Figure 1.1: Conceptual graph-to-trade pipeline. {8}
- Figure 1.2: Research questions and benchmark dimensions. {11}
- Figure 2.1: Layer-wise graph-convolutional update of node embeddings. {14}
- Figure 3.1: General financial graph-learning pipeline. {16}
- Figure 3.2: Pre-holdout exploratory overview of the three-asset study universe. {17}
- Figure 3.3: Graph input representation. {18}
- Figure 3.4: Split design and Final Holdout difference. {23}
- Figure 3.5: Triple barrier construction. {24}
- Figure 3.6: Chronological fold design (chronological walk-forward folds). {27}
- Figure 4.1: Comparison of the Architectures. {30}
- Figure 4.2: Detailed architecture of the basegraph family. {31}
- Figure 4.3: Detailed architecture of the multigraph family. {33}
- Figure 4.4: Detailed memorygraph architecture with recurrent node and edge memory. {35}
- Figure 5.1: Benchmark overview by frequency, graph family, and operator. {37}
- Figure 5.2: Cumulative gross PnL paths for representative final-holdout models. {38}
- Figure 5.3: Cumulative net PnL paths for representative final-holdout models. {39}
- Figure 5.4: Gross vs net PnL for 1sec models. {41}
- Figure 5.5: One-second cumulative gross-versus-net PnL paths for memory-gnn-conv and base-gnn-mpnn. {41}
- Figure 5.6: last-fold-M versus final-refit-M as deployment-oriented model states. {44}

List of Tables

Table 3.1: Implemented node features. {19}

Table 3.2: Frequency-specific experimental regimes and validation settings. {22}

Table 4.1: Controlled architectural comparison across the three model families. {29}

Table 5.1: Time spent on GNN experiments on Google Cloud Platform in hours. {36}

Table 5.1: last-fold-M benchmark overview across all model-frequency configurations. {37}

Table 5.2: The best five-minute model states deployment-state comparison. {45}

Table 5.3: The best one-minute model deployment-state comparison. {45}

Table 5.4: The most gross-profitable one-second model deployment-state comparison. {45}

Table 5.5: The most direction-oriented five-minute model deployment-state comparison. {46}

Table 5.6: Summary of answers to the research questions. {48}

Table 5.7: Summary of hypothesis assessment. {48}

Table A.1: Model-family and operator mapping. {59}

Table A.2: Frequency and task configuration. {59}

Table A.3: Shared target and loss configuration. {59}

Table B.1: Final-holdout alignment records by run. {60}

Table B.2: Frequency-level final-holdout summary. {60}

Table C.1: Best or least-negative last-fold-M model by frequency. {61}

Table C.2: Full last-fold-M diagnostic table. {61}

Table C.3: Full final-refit-M diagnostic table. {62}

Table C.4: Main high-frequency turnover example. {62}

Documentation table of AI-based tools

AI-based tools	Intended use	Prompt, source, page, paragraph...
ChatGPT (GPT-5)	Initial scientific works exploration (as an additional source of papers)	“Provide 50 papers relevant to GNN, especially for time series modeling or financial markets. The papers should be from good journals or highly mentioned by other researchers. For each paper you should give me the link to access it.”
Copilot (GPT-5.4)	Code draft for figures. (but the final figures highly manually changed)	“I gave you a description of some of the figures as a Markdown file. Create a draft of the code that can generate plots based on these descriptions - you have access to initial csv files.” Input: figures_plan.md
ChatGPT (5.5)	Grammar and spelling	“Please list issues with spelling and grammar in the following document” Entire document
Codex (GPT-5.5)	List of Figures	“Create a list that mentions all the figures in this paper. Format: Figure N: name. {page}” Entire document
Codex (GPT-5.5)	List of Tables	“Create a list that mentions all the tables in this paper. Format: Table N: name. {page}” Entire document

Appendix A. Model Configurations

This appendix shows the main benchmark configuration in a compact form. It provides a quick reference for the model-family, operator, frequency, and target-design choices used throughout the thesis. Table A.1 maps model labels to families and operators, Table A.2 lists the frequency-specific task design, and Table A.3 summarizes the shared target and loss configuration.

Table A.1. Model-family and operator mapping.

Benchmark label	Family	Graph operator	Main architectural distinction
base-gnn-conv	basegraph	adaptive_conv	Early relation fusion with Conv-style graph processing.
base-gnn-mpnn	basegraph	adaptive_mpnn	Early relation fusion with MPNN-style message passing.
multi-gnn-conv	multigraph	dynamic_rel_conv	Relation-specific graph pathways with Conv-style processing.
multi-gnn-mpnn	multigraph	dynamic_edge_mpnn	Relation-specific graph pathways with MPNN-style processing.
memory-gnn-conv	memorygraph	conv	Stateful recurrent node-edge memory with Conv-style graph updates.
memory-gnn-mpnn	memorygraph	mpnn	Stateful recurrent node-edge memory with MPNN-style graph updates.

Table A.2. Frequency and task configuration.

Frequency	Lookback	Forecast horizon	Working data slice	Final holdout fraction	CV folds	Interpretation
5min	30 minutes = 6 bars	5 minutes = 1 bar	0.0-0.9 of the full series	0.10	4	Strict shared-task benchmark.
1min	30 minutes = 30 bars	5 minutes = 5 bars	0.0-0.9 of the full series	0.10	4	Strict shared-task benchmark.
1sec	2 minutes = 120 bars	2 minutes = 120 bars	0.5-0.9 of the full series	0.225	2	Frequency-adapted high-frequency stress test.

Table A.3. Shared target and loss configuration.

Component	Configuration used in the benchmark
Target asset	ETH
Context assets	ADA and BTC
Target construction	Volatility-scaled triple-barrier framework
Output heads	trade_logit, dir_logit, return_pred, exit_type_logit, tte_pred
Loss weights	loss_w_trade = 0.35, loss_w_dir = 0.65, loss_w_ret = 0.15, loss_w_utility = 0.85, loss_w_exit_type = 0.05, loss_w_tte = 0.03
Main economic metric	net_pnl
Cost proxy	cost_bps_unit = 0.0003

Appendix B. Final Holdout Alignment Table

Source: [github](#) final_runs/*/splits/split_summary.json, split_indices.npz, resolved_config.yaml.

Table B.1 reports the run-level alignment records.

Table B.2 summarizes the same alignment at frequency level.

Table B.1. Final-holdout alignment records by run.

run	freq	slice	holdout_frac	holdout_start_utc	holdout_end_utc	holdout_n	effective_holdout_full_frac	delta_vs_1min
1min-base-gnn-conv	1min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:07:00	1543	0.809977-0.899942	start 0s / end 0s
1min-base-gnn-mpnn	1min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:07:00	1543	0.809977-0.899942	start 0s / end 0s
1min-memory-gnn	1min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:07:00	1543	0.809977-0.899942	start 0s / end 0s
1min-multi-gnn-conv	1min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:07:00	1543	0.809977-0.899942	start 0s / end 0s
1min-multi-gnn-mpnn	1min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:07:00	1543	0.809977-0.899942	start 0s / end 0s
1sec-base-gnn-conv	1sec	0.5-0.9	0.225	2021-04-17T03:25:34	2021-04-18T05:10:01	92668	0.810000-0.899999	start 34s / end 181s
1sec-base-gnn-mpnn	1sec	0.5-0.9	0.225	2021-04-17T03:25:34	2021-04-18T05:10:01	92668	0.810000-0.899999	start 34s / end 181s
1sec-memory-gnn-conv	1sec	0.5-0.9	0.225	2021-04-17T03:25:34	2021-04-18T05:10:01	92668	0.810000-0.899999	start 34s / end 181s
1sec-memory-gnn-mpnn	1sec	0.5-0.9	0.225	2021-04-17T03:25:34	2021-04-18T05:10:01	92668	0.810000-0.899999	start 34s / end 181s
1sec-multi-gnn-conv	1sec	0.5-0.9	0.225	2021-04-17T03:25:34	2021-04-18T05:10:01	92668	0.810000-0.899999	start 34s / end 181s
1sec-multi-gnn-mpnn	1sec	0.5-0.9	0.225	2021-04-17T03:25:34	2021-04-18T05:10:01	92668	0.810000-0.899999	start 34s / end 181s
5min-base-gnn	5min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:05:00	309	0.809883-0.899708	start 0s / end -120s
5min-memory-gnn	5min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:05:00	309	0.809883-0.899708	start 0s / end -120s
5min-multi-gnn	5min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:05:00	309	0.809883-0.899708	start 0s / end -120s

Table B.2. Frequency-level final-holdout summary.

freq	runs	shared_holdout_start_utc	shared_holdout_end_utc	shared_holdout_n	note
1min	5	2021-04-17T03:25:00	2021-04-18T05:07:00	1543	reference window
1sec	6	2021-04-17T03:25:34	2021-04-18T05:10:01	92668	starts 34s later than 1min, ends 181s later
5min	3	2021-04-17T03:25:00	2021-04-18T05:05:00	309	same start as 1min, ends 120s earlier

Appendix C. Additional Benchmark Tables

This appendix records supplementary diagnostic summaries that support the benchmark interpretation. Even more extensive tables can be found on Google Spreadsheets: [link](#).

Table C.1 summarizes the best or least-negative **last-fold-M** model by frequency.

Table C.2 reports the full **last-fold-M** diagnostic metrics.

Table C.3 reports the corresponding **final-refit-M** diagnostics.

Table C.4 isolates the high-frequency turnover example.

*Table C.1. Best or least-negative **last-fold-M** model by frequency.*

Frequency	Model	gross_pnl	net_pnl	n_trades	Interpretation
5min	base-gnn-conv	0.028156	0.020356	26	Best positive shared-task result.
1min	base-gnn-conv	0.059694	0.020094	132	Best 1min result, with larger gross signal but higher turnover.
1sec	base-gnn-mpnn	0.052679	-0.065821	395	Least negative 1sec model; still not deployment-grade after costs.

*Table C.2. Full **last-fold-M** diagnostic table.*

Frequency	Model	gross_pnl	net_pnl	pnl_per_trade	n_trades	trade_rate	sign_accuracy	win_rate	sharpe_ratio	dir_auc	trade_auc	RMSE
5min	base-gnn-conv	0.028156	0.020356	0.000783	26	0.084142	0.692308	0.692308	1.914418	0.617105	0.700447	0.028769
5min	base-gnn-mpnn	0.014415	0.006915	0.000277	25	0.080906	0.560000	0.560000	0.508262	0.614912	0.727631	0.038402
5min	multi-gnn-conv	0.012758	0.001958	0.000054	36	0.116505	0.555556	0.555556	0.127740	0.616667	0.672097	0.041531
5min	multi-gnn-mpnn	0.002941	-0.009359	-0.000228	41	0.132686	0.487805	0.487805	-0.594956	0.625439	0.707304	0.029870
5min	memory-gnn-conv	0.009459	0.004359	0.000256	17	0.055016	0.588235	0.588235	0.366933	0.611842	0.734196	0.044066
5min	memory-gnn-mpnn	-0.012463	-0.037363	-0.000450	83	0.268608	0.445783	0.445783	-1.607095	0.537719	0.726026	0.024487
1min	base-gnn-conv	0.059694	0.020094	0.000152	132	0.085548	0.583333	0.568182	0.644455	0.525927	0.648157	0.002269
1min	base-gnn-mpnn	-0.024239	-0.078539	-0.000434	181	0.117304	0.469613	0.436464	-2.489121	0.504512	0.642097	0.003080
1min	multi-gnn-conv	0.025767	-0.007833	-0.000070	112	0.072586	0.544643	0.535714	-0.288013	0.541996	0.634787	0.005728
1min	multi-gnn-mpnn	-0.003371	-0.035771	-0.000331	108	0.069994	0.490741	0.481481	-1.294441	0.528405	0.645004	0.005223
1min	memory-gnn-conv	-0.031247	-0.078947	-0.000497	159	0.103046	0.459119	0.452830	-2.410248	0.479399	0.638928	0.002455
1min	memory-gnn-mpnn	0.033605	0.009305	0.000115	81	0.052495	0.567901	0.555556	0.380381	0.529844	0.635665	0.010463
1sec	base-gnn-conv	0.139515	-0.094185	-0.000121	779	0.008406	0.684211	0.680359	-6.506930	0.599753	0.841804	0.000592
1sec	base-gnn-mpnn	0.052679	-0.065821	-0.000167	395	0.004263	0.663291	0.658228	-8.430534	0.599093	0.848558	0.000569
1sec	multi-gnn-conv	0.081710	-0.108790	-0.000171	635	0.006852	0.656693	0.650394	-10.771267	0.597838	0.839544	0.000539
1sec	multi-gnn-mpnn	0.079777	-0.080723	-0.000151	535	0.005773	0.680374	0.676636	-7.960091	0.600538	0.868370	0.000504
1sec	memory-gnn-conv	0.412032	-1.163268	-0.000222	5251	0.056665	0.584841	0.584079	-28.865252	0.588785	0.490050	0.001247
1sec	memory-gnn-mpnn	0.223788	-0.280512	-0.000167	1681	0.018140	0.662106	0.660916	-17.034172	0.596713	0.863699	0.000578

Table C.3. Full **final-refit-M** diagnostic table.

Frequency	Model	gross_pnl	net_pnl	pnl_per_trade	n_trades	trade_rate	sign_accuracy	win_rate	sharpe_ratio	dir_auc	trade_auc	RMSE
5min	base-gnn-conv	0.017570	0.011270	0.000537	21	0.067961	0.619048	0.619048	0.988114	0.630702	0.721795	0.021324
5min	base-gnn-mpnn	0.020722	0.008722	0.000218	40	0.129450	0.575000	0.575000	0.545576	0.636842	0.715911	0.016092
5min	multi-gnn-conv	0.061167	0.041367	0.000627	66	0.213592	0.681818	0.681818	2.182646	0.703947	0.720920	0.017109
5min	multi-gnn-mpnn	0.008092	0.002992	0.000176	17	0.055016	0.588235	0.588235	0.309342	0.634211	0.707158	0.021148
5min	memory-gnn-conv	0.029075	-0.000325	-0.000003	98	0.317152	0.540816	0.530612	-0.013141	0.631140	0.722914	0.016015
5min	memory-gnn-mpnn	0.017905	-0.014495	-0.000134	108	0.349515	0.527778	0.518519	-0.548738	0.544298	0.737259	0.014369
1min	base-gnn-conv	0.007198	-0.012002	-0.000188	64	0.041478	0.484375	0.484375	-0.555220	0.524286	0.635712	0.003175
1min	base-gnn-mpnn	0.034198	0.006298	0.000068	93	0.060272	0.548387	0.548387	0.234208	0.513907	0.655608	0.003194
1min	multi-gnn-conv	0.001944	-0.037056	-0.000285	130	0.084251	0.492308	0.492308	-1.153049	0.543084	0.641036	0.004606
1min	multi-gnn-mpnn	0.021735	-0.011565	-0.000104	111	0.071938	0.540541	0.540541	-0.386487	0.522804	0.627002	0.003473
1min	memory-gnn-conv	-0.011482	-0.049882	-0.000390	128	0.082955	0.484375	0.476562	-1.816363	0.525231	0.619879	0.002149
1min	memory-gnn-mpnn	0.049531	-0.055769	-0.000159	351	0.227479	0.524217	0.504274	-1.323040	0.516055	0.562599	0.002287
1sec	base-gnn-conv	0.262794	-0.264606	-0.000151	1758	0.018971	0.668942	0.665529	-11.677533	0.590194	0.828703	0.000511
1sec	base-gnn-mpnn	0.108520	-0.089180	-0.000135	659	0.007111	0.688923	0.682853	-7.092366	0.598878	0.842822	0.000499
1sec	multi-gnn-conv	0.091421	-0.073579	-0.000134	550	0.005935	0.703636	0.694545	-8.415791	0.597449	0.843536	0.000494
1sec	multi-gnn-mpnn	0.057127	-0.078773	-0.000174	453	0.004888	0.657837	0.651214	-9.356826	0.600348	0.865638	0.000519
1sec	memory-gnn-conv	0.443031	-0.954969	-0.000205	4660	0.050287	0.604506	0.603219	-25.126196	0.592186	0.852874	0.000529
1sec	memory-gnn-mpnn	0.198876	-0.202524	-0.000151	1338	0.014439	0.671898	0.668909	-13.431875	0.593766	0.865245	0.000590

Table C.4. Main high-frequency turnover example.

Frequency	Model	gross_pnl	net_pnl	n_trades	Approximate cumulative cost	Interpretation
1sec	memory-gnn-conv	0.412032	-1.163268	5251	1.5753	Strong gross signal is overwhelmed by excessive turnover and transaction costs.